

Projektarbeit Machine Learning

Von Nico Berns, Marvin Jordan, Luca Martens und
Claudio Cosentino

CNN

Convolutional Neural Network

Roadmap CNN



Konzeption



Mathematik



CIFAR 10



Modellaufbau
und Training



Evaluierung

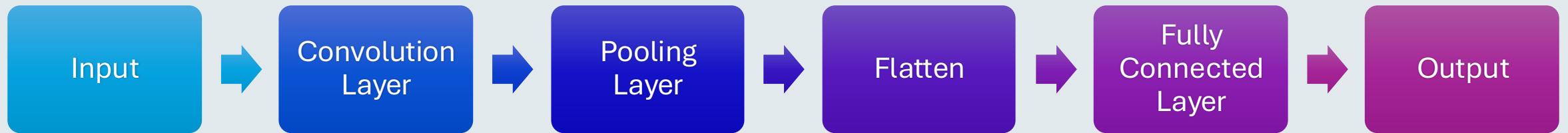


Hyperparameter
Tuning



Ergebnisse

Konzeption



Input

- Tensor im Format HxWxC z. B. 32x32x3 bei den CIFAR 10 Bildern
- Jeder Pixel hat somit 3 8-Bit Werte:
 - Rot: [0, 255]
 - Grün: [0, 255]
 - Blau: [0, 255]
- Damit das neuronale Netz stabiler lernen kann werden die Pixel zwischen [0, 1] normiert

1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
0	0	0	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1

Convolution Layer

- Filter/Kernel z. B. 3x3 Matrix
- Kernel wird über den Input gefaltet/geschoben
 - Stride beschreibt wieviel Pixel der Kernel weiter geschoben wird
 - Channel sind die Farbkanäle des Inputs
 - Padding ergänzt um den Input einen Rahmen von 0
- Kernel enthält Gewichte
- Verschiedene Gewichte => Verschiedene Ergebnisse
- Convolution Formel:
 - $Y(i, j) = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} X(i + u, j + v) * W(u, v) + b$
 - Y = Ausgabe-Feature-Map
 - X = Eingabe
 - W = Kernel
 - b = Bias

-1	0	1
-1	0	1
-1	0	1

Vertikaler Filter

-1	-1	-1
0	0	0
1	1	1

Horizontaler Filter

0	-1	0
-1	4	-1
0	-1	0

Laplace Filter

Convolution Layer

1	1	1	0	0	0
1	1	1	0	0	0
1	1	1	0	0	0
0	0	0	1	1	1
0	0	0	1	1	1
0	0	0	1	1	1

Input: $(i * i) = (6 * 6)$

*

-1	0	1
-1	0	1
-1	0	1

Filter: $(f * f) = (3 * 3)$

=

0	-3	-3	0
0	-1	-1	0
0	1	1	0
0	3	3	0

Feature Map: $(i - f + 1 * i - f + 1) = (4 * 4)$

Output [0,0] = $(1 * -1) + (1 * 0) + (1 * 1) + (1 * -1) + (1 * 0) + (1 * 1) + (1 * -1) + (1 * 0) + (1 * 1) = 0$

Aktivierungsfunktion

- Wird benötigt, um nichtlineare Zusammenhänge zu lernen
- Arten von Aktivierungsfunktionen:
 - Linear: $f(x) = x$
 - Sigmoid: $f(x) = \frac{1}{1+e^{-x}}$
 - Tanh: $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
 - ReLU: $f(x) = \max(0, x)$
 - Softmax: $f(x) = \frac{e^{x_i}}{\sum_j e^{x_j}}$
- Verwendet:
 - Rectified Linear Unit
 - Softmax

0	0	0	0
0	0	0	0
0	1	1	0
0	3	3	0

Pooling Layer

- Reduktion der räumlichen Größe
- Verwendung z. B. einer 2x2 Matrix
- Repräsentativster Wert wird übernommen
 - Max Pooling: nimmt den größten Wert in dem Bereich
 - Average Pooling: nimmt den Mittelwert des Bereiches
- Weniger Rechenaufwand
- Max Pooling Formel:
 - $Y(i, j) = \max_{(u, v) \in \Omega} X(i + u, j + v)$
 - Ω = Pooling Fenster z. B. 2x2
- Average Pooling Formel:
 - $Y(i, j) = \frac{1}{|\Omega|} \sum_{(u, v) \in \Omega} X(i + u, j + v)$

0	0	0	0
0	0	0	0
0	1	1	0
0	3	3	0



0	0
3	3

Feature Map: $\left(\frac{f}{p} * \frac{f}{p}\right) = (2 * 2)$

Feature Map: $(f * f) = (4 * 4)$

Pooling Matrix: $(p * p) = (2 * 2)$

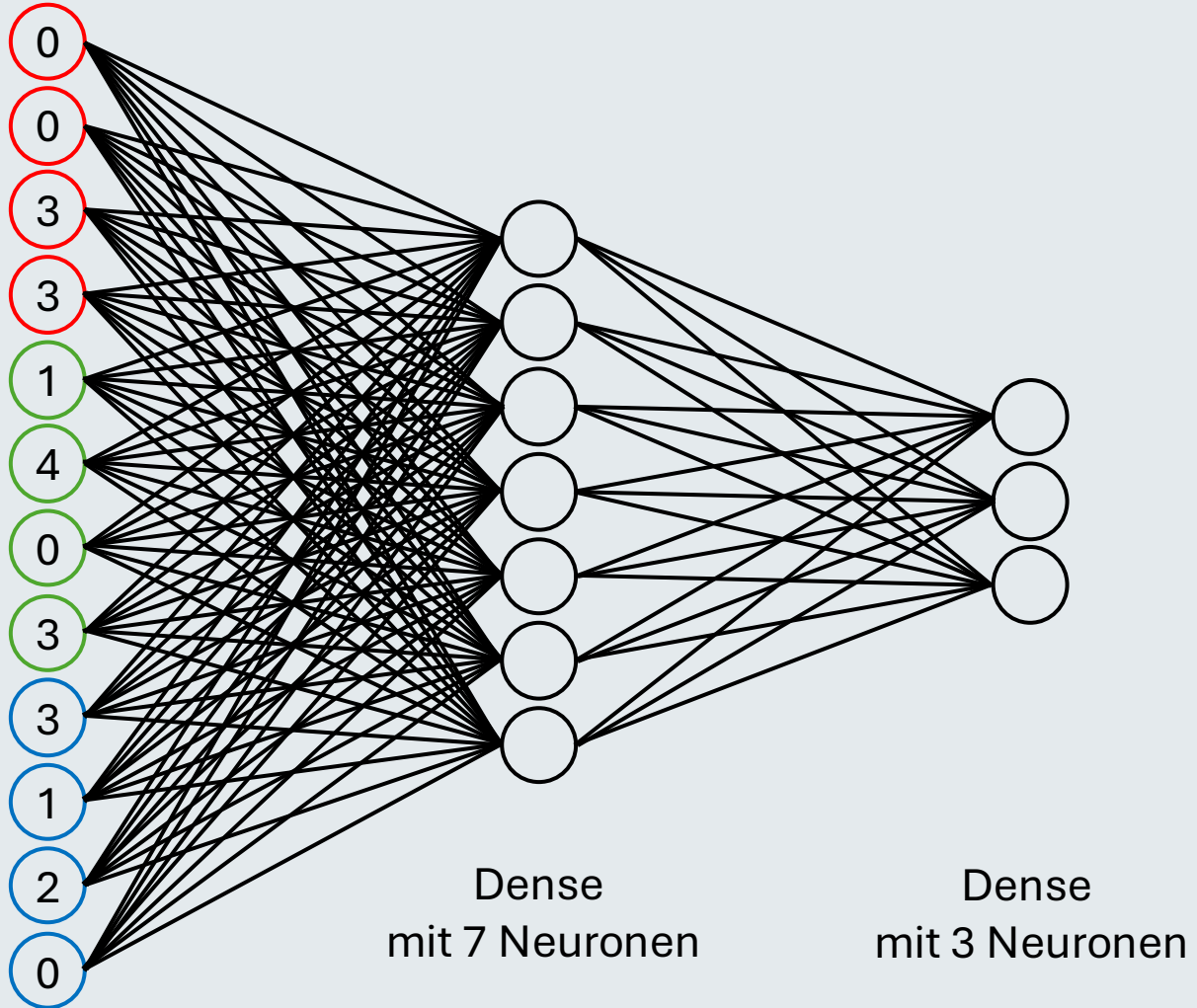
Fully Connected Layer

0	0
3	3

1	4
0	3

3	1
2	0

Flatten



Visuelles Beispiel

Input



Original
1322x1322



Graustufen
1322x1322

Vertikal

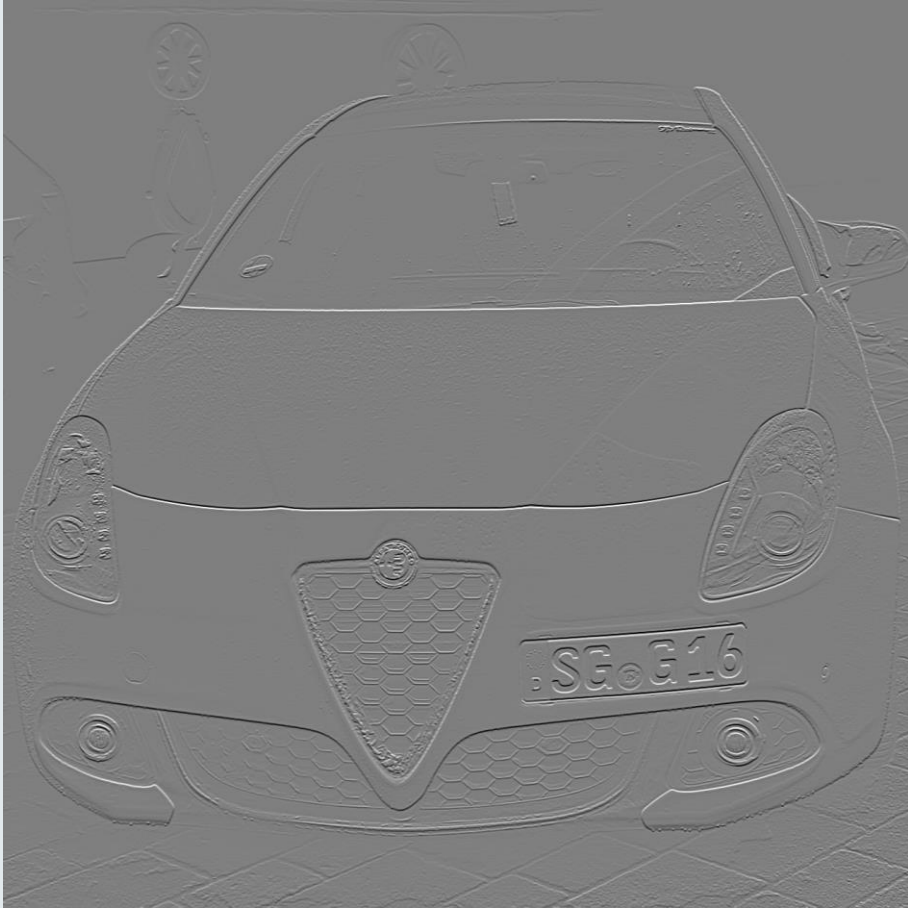


Vertical Convolution, Padded
1322x1322



Vertical Convolution, Padded, Pooling 2x2
661x661

Horizontal

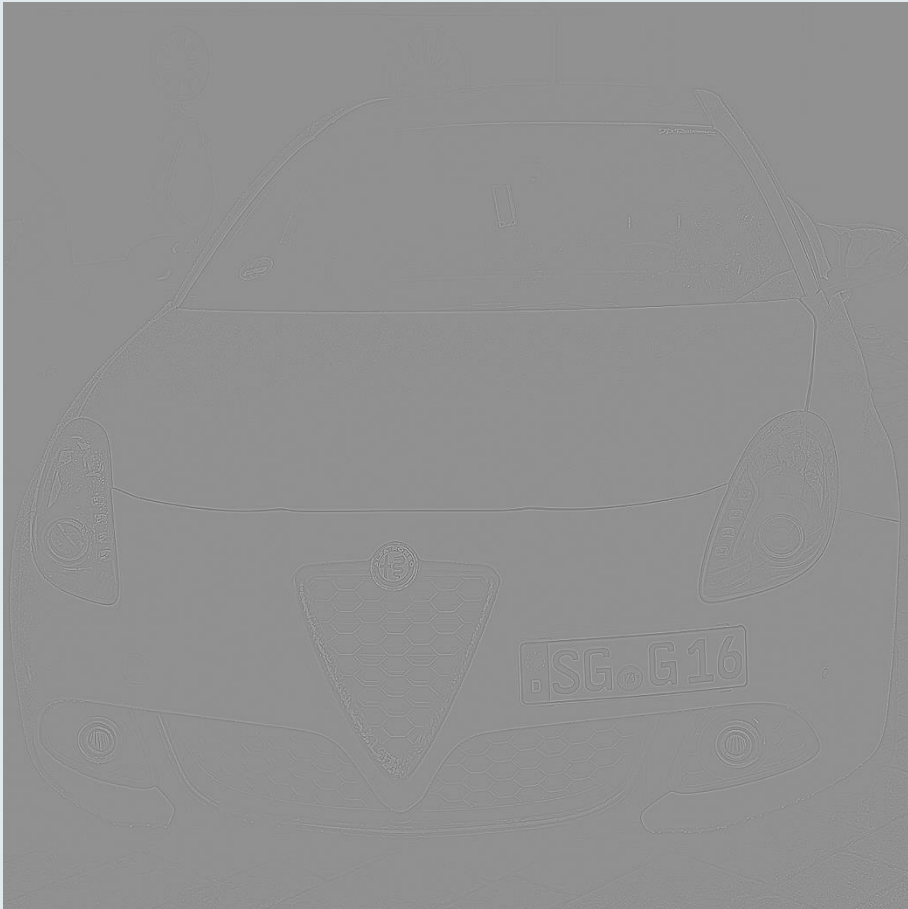


Horizontal Convolution, Padded
1322x1322



Horizontal Convolution, Padded, Pooling 2x2
661x661

Laplace



Laplace Convolution, Padded
1322x1322



Laplace Convolution, Padded, Pooling 2x2
661x661

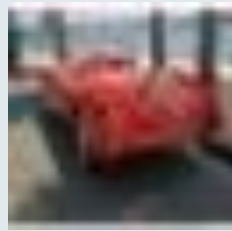
CIFAR-10

CIFAR-10

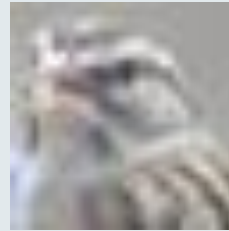
- Datensatz beinhaltet 60.000 Bilder mit jeweils 32x32 Pixeln
- Unterteilt in 10 Klassen mit je 6.000 Bildern
- Datensatz ist in fünf Trainings-Batches und ein Test-Batch mit jeweils 10.000 Bildern eingeteilt



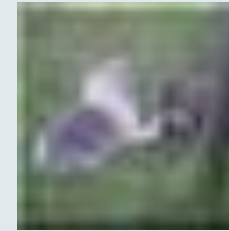
Airplane



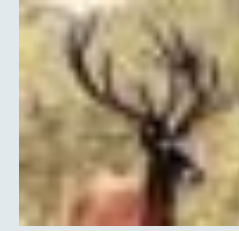
Automobile



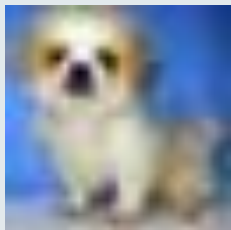
Bird



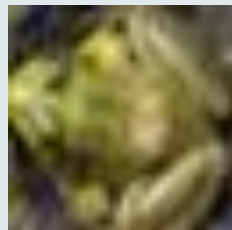
Cat



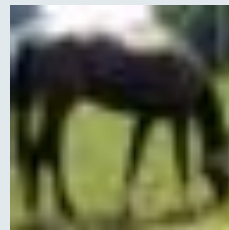
Deer



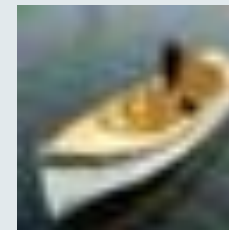
Dog



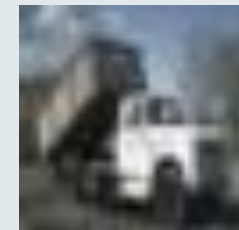
Frog



Horse



Ship



Truck

Modellaufbau & Training



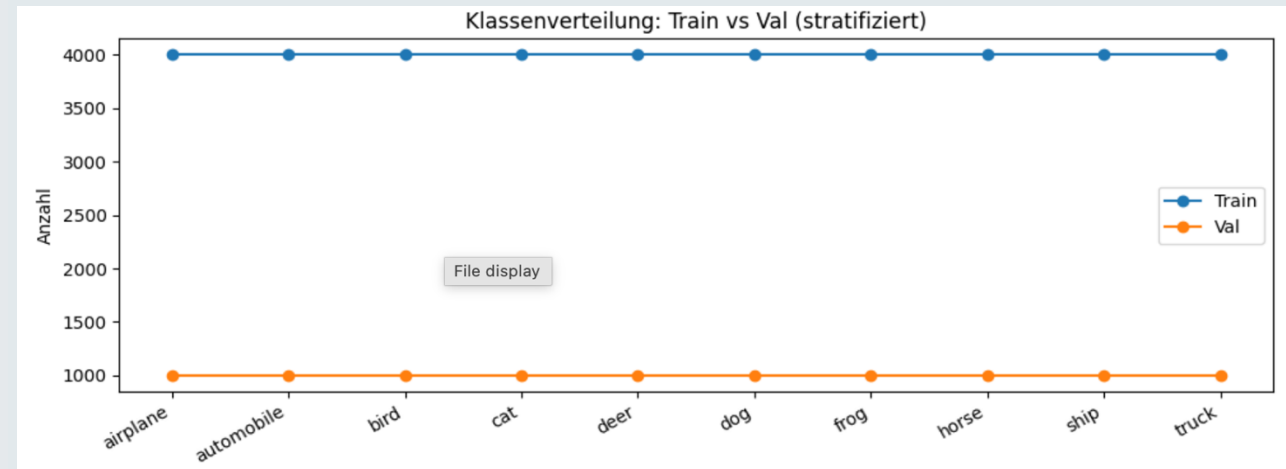
```
1 x_train_full = x_train_full.astype("float32") / 255.0
2 x_test = x_test.astype("float32") / 255.0
3
4 print("Min/Max train:", x_train_full.min(), x_train_full.max())
5 print("Min/Max test:", x_test.min(), x_test.max())
```

- Normierung auf Wertebereich [0 - 1]
- Ausgabe des Min- und Max-Wertes

Modellaufbau & Training

```
1 x_train, x_val, y_train, y_val = train_test_split(
2     x_train_full, y_train_full,
3     test_size=0.2,
4     random_state=42,
5     stratify=y_train_full
6 )
7
8 print("Train:", x_train.shape, y_train.shape)
9 print("Val: ", x_val.shape, y_val.shape)
10 print("Test: ", x_test.shape, y_test.shape)
11
12 train_counts = np.bincount(y_train, minlength=10)
13 val_counts = np.bincount(y_val, minlength=10)
14
15 plt.figure(figsize=(10,4))
16 plt.plot(class_names, train_counts, marker="o", label="Train")
17 plt.plot(class_names, val_counts, marker="o", label="Val")
18 plt.title("Klassenverteilung: Train vs Val (stratifiziert)")
19 plt.ylabel("Anzahl")
20 plt.xticks(rotation=30, ha="right")
21 plt.legend()
22 plt.tight_layout()
23 plt.show()
```

- Aufteilung der Trainingsdaten in Trainings- und Validierungsdaten
- Ausgabe der Aufteilung zur Überprüfung
- Zählen der Trainings- und Validierungsbilder je Label
- Ausgabe der Ergebnisse in einer Grafik



Modellaufbau & Training

```
1  tf.random.set_seed(42)
2  np.random.seed(42)
3
4  model = keras.Sequential([
5      layers.Input(shape=(32, 32, 3)),
6
7      layers.Conv2D(32, (3,3), padding="same", activation="relu"),
8      layers.Conv2D(32, (3,3), padding="same", activation="relu"),
9      layers.MaxPooling2D((2,2)),
10     layers.Dropout(0.25),
11
12     layers.Conv2D(64, (3,3), padding="same", activation="relu"),
13     layers.Conv2D(64, (3,3), padding="same", activation="relu"),
14     layers.MaxPooling2D((2,2)),
15     layers.Dropout(0.25),
16
17     layers.Flatten(),
18     layers.Dense(256, activation="relu"),
19     layers.Dropout(0.5),
20     layers.Dense(10, activation="softmax")
21 ])
22
23 model.summary()
```

- Definierung des sequenziellen Durchlaufs der Layer
- Am Ende mit einer Zusammenfassung aller Layer

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 32, 32, 32)	896
conv2d_5 (Conv2D)	(None, 32, 32, 32)	9,248
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 32)	0
dropout_3 (Dropout)	(None, 16, 16, 32)	0
conv2d_6 (Conv2D)	(None, 16, 16, 64)	18,496
conv2d_7 (Conv2D)	(None, 16, 16, 64)	36,928
max_pooling2d_3 (MaxPooling2D)	(None, 8, 8, 64)	0
dropout_4 (Dropout)	(None, 8, 8, 64)	0
flatten_1 (Flatten)	(None, 4096)	0
dense_2 (Dense)	(None, 256)	1,048,832
dropout_5 (Dropout)	(None, 256)	0
dense_3 (Dense)	(None, 10)	2,570

Total params: 1,116,970 (4.26 MB)

Trainable params: 1,116,970 (4.26 MB)

Non-trainable params: 0 (0.00 B)

Modellaufbau & Training



```
1 model.compile(  
2     optimizer=keras.optimizers.Adam(learning_rate=1e-3),  
3     loss="sparse_categorical_crossentropy",  
4     metrics=["accuracy"]  
5 )
```



```
1 history = model.fit(  
2     x_train, y_train,  
3     validation_data=(x_val, y_val),  
4     epochs=10,  
5     batch_size=64,  
6     verbose=1  
7 )
```

Evaluierung – Loss & Accuracy

```
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_acc:.4f}")
```

Test Loss

→

durchschnittlicher
Fehler?



71,35%

Test Accuracy

→

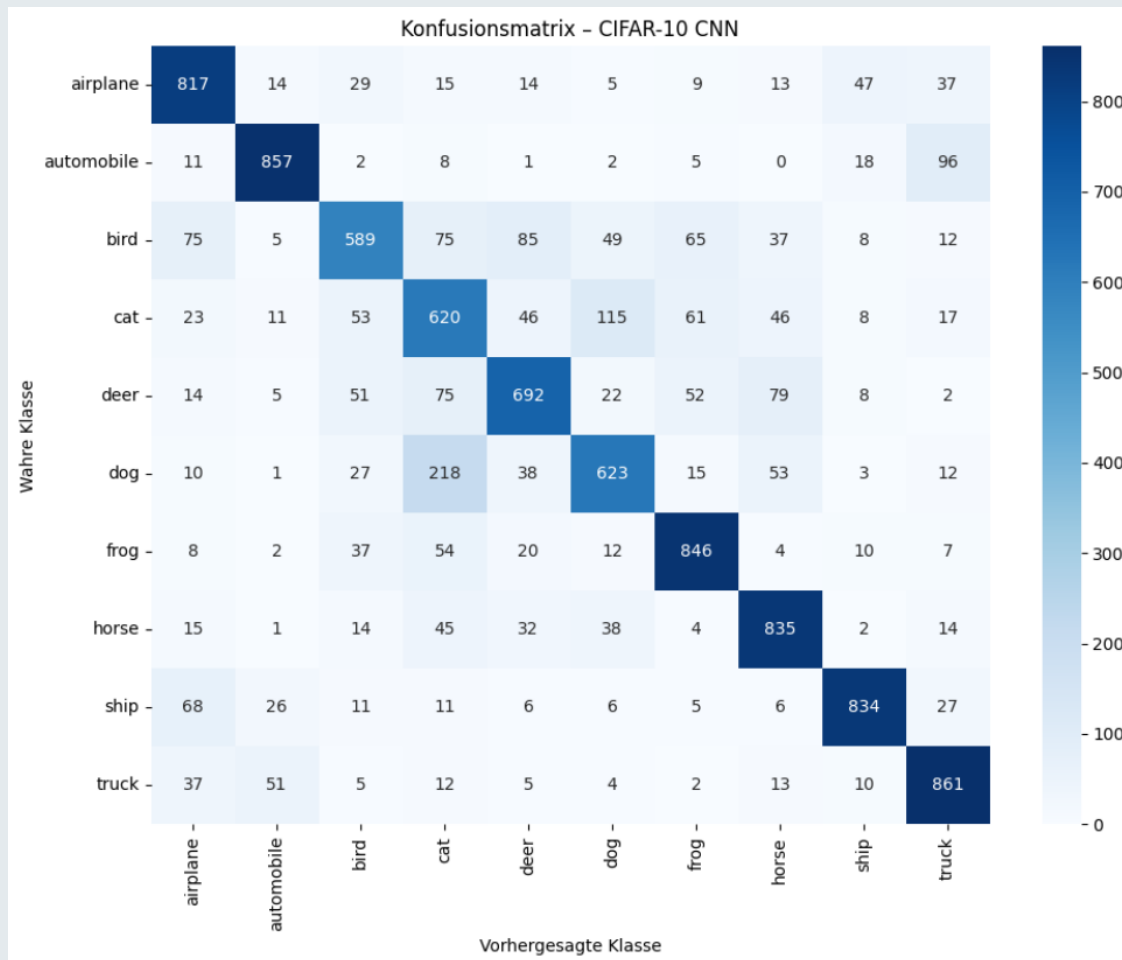
Wie oft korrekt
klassifiziert?



74,87%

Ziel: Test Loss **niedrig** und Accuracy **hoch**!

Evaluierung - Konfusionsmatrix



- Konfusionsmatrix zeigt Verhältnis von vorhergesagter zu wahrer Klasse
 - Diagonale zeigt korrekt klassifizierte Bilder
 - Hohe Werte auf der Diagonalen
→ Gute Erkennung vieler Klassen
 - Systematische Fehlklassifikationen erkennbar
- Zeigt detailliert, welche Klassen korrekt erkannt wurden.

Evaluierung - Konfusionsmatrix

```
y_pred_prob = model.predict(x_test, verbose=0)
y_pred = np.argmax(y_pred_prob, axis=1)

from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

import seaborn as sns

plt.figure(figsize=(10,8))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names
)
plt.xlabel("Vorhergesagte Klasse")
plt.ylabel("Wahre Klasse")
plt.title("Konfusionsmatrix - CIFAR-10 CNN")
plt.tight_layout()
plt.show()
```



[1] `model.predict(x_test,...)`
→ berechnet für jedes Testbild die Wahrscheinlichkeiten aller Klassen

[2] `np.argmax(...,axis=1)`
→ wählt für jedes Bild die Klasse mit der höchsten vorhergesagten Wahrscheinlichkeit

[6] `confusion_matrix(y_test,y_pred)`
→ Vergleich von wahrer und vorhergesagter Klasse

[10] `sns.heatmap(cm,...)`
→ visualisiert die Konfusionsmatrix als farbige Heatmap

[14] `cmap = "Blues"`
→ macht den Blauverlauf

Hyperparameter Tuning

gewählte Hyperparameter:

```
# Basismodell

batch_size = 64,
learning_rate = 1e-3,
layers.Conv2D(32, (3,3), padding="same", activation="relu"),
```

- batch_size
 - bestimmt, wie viele Bilder gleichzeitig verarbeitet werden, bevor Gewichte des Modells aktualisiert werden.
- learning_rate
 - wie stark die Gewichte bei jedem Trainingsschritt angepasst werden.
- layers.Conv2D(...)
 - Anzahl bestimmt, wie viele unterschiedliche Merkmale das CNN in einer Schicht lernen kann.

Hyperparameter Tuning – Variante 1

```
# Variante 1

batch_size = 128, # von 64 auf 128 erhöht
learning_rate = 1e-3,
layers.Conv2D(32, (3,3), padding="same", activation="relu"),
```

- Erhöhung der batch_size von 64 auf 128.
- Der Rest bleibt unverändert.

Was ist das Ergebnis?

Ergebnisse des Basismodells

Test Loss → 71,35%
Test Accuracy → 74,87%

Ergebnisse der 1. Variante

Test Loss → 72,68% | ↗ 1,33%
Test Accuracy → 74,18% | ↘ 0,69%

Hyperparameter Tuning – Variante 2

```
# Variante 2

batch_size = 64,
learning_rate = 5e-4, # von 1e-3 auf 5e-4 verändert
layers.Conv2D(32, (3,3), padding="same", activation="relu"),
```

- Änderung der learning_rate von 1e-3 auf 5e-4.
- Der Rest bleibt unverändert.

Was ist das Ergebnis?

Ergebnisse des Basismodells

Test Loss → 71,35%
Test Accuracy → 74,87%

Ergebnisse der 2. Variante

Test Loss → 73,10% | ↗ 1,75%
Test Accuracy → 74,99% | ↗ 0,12%

Hyperparameter Tuning – Variante 3

```
# Variante 3  
  
batch_size = 64,  
learning_rate = 1e-3,  
layers.Conv2D(64, (3,3), padding="same", activation="relu"), # Anpassung von 32 auf 64
```

- Anpassung von layers.Conv2D von 32 auf 64.
- Der Rest bleibt unverändert.

Was ist das Ergebnis?

Ergebnisse des Basismodells

Test Loss → 71,35%
Test Accuracy → 74,87%

Ergebnisse der 3. Variante

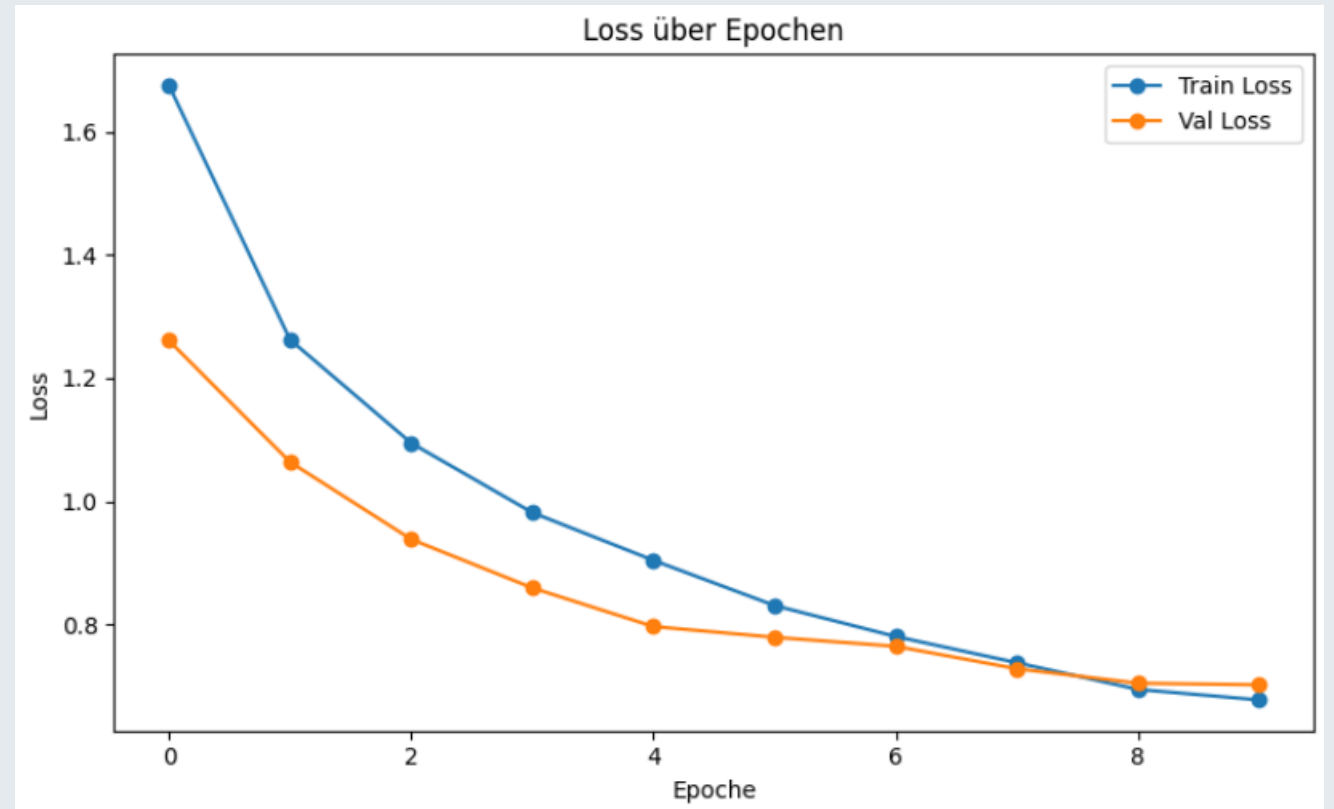
Test Loss → 71,20% | ↘ 0,15%
Test Accuracy → 75,68% | ↗ 0,81%

Ergebnisse – Loss

```
hist = history.history

plt.figure(figsize=(8,5))
plt.plot(hist["loss"], marker="o", label="Train Loss")
plt.plot(hist["val_loss"], marker="o", label="Val Loss")
plt.title("Loss über Epochen")
plt.xlabel("Epoche")
plt.ylabel("Loss")
plt.legend()
plt.tight_layout()
plt.show()
```

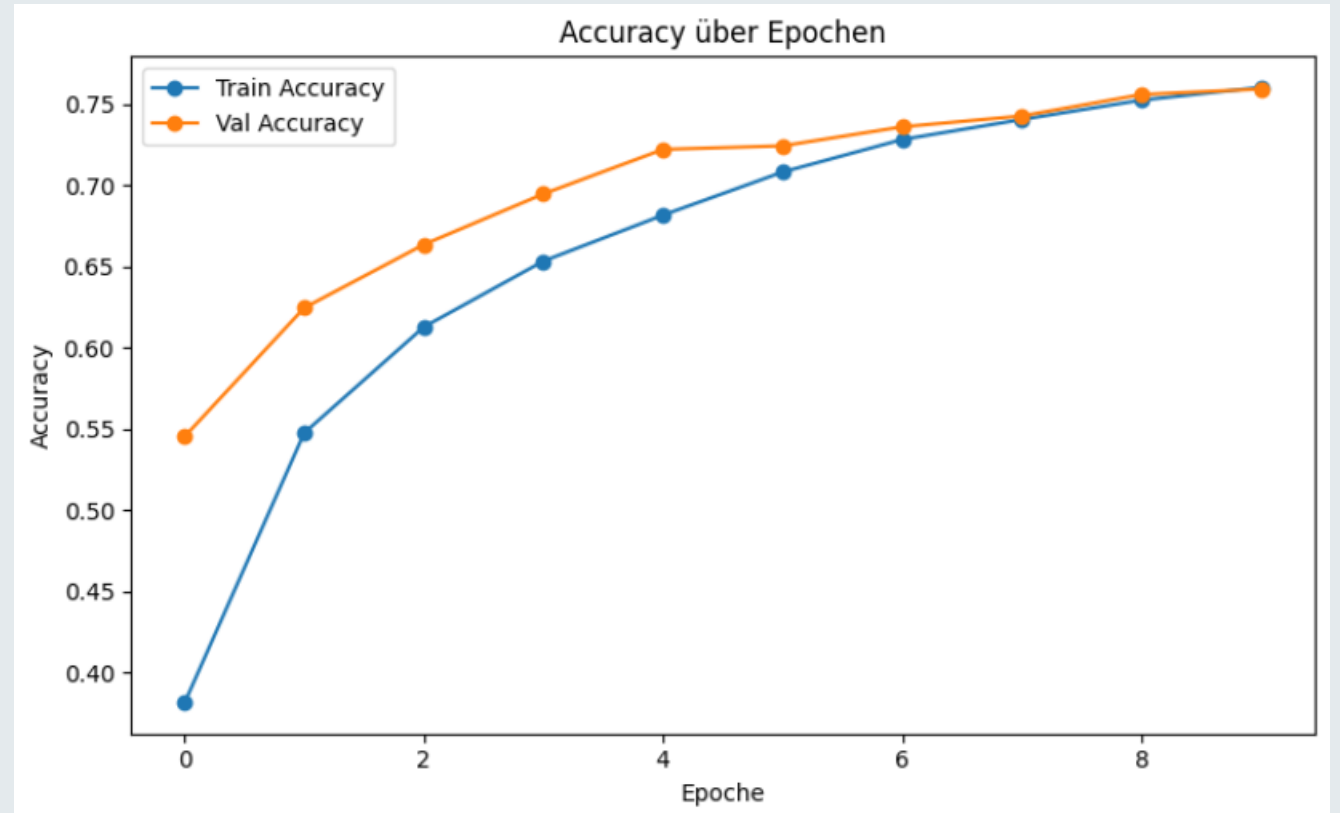
- Plotten der Loss-Werte über 10 Epochen
- Die Trainings- und Validierungsverläufe zeigen eine stabile Konvergenz des Modells.



Ergebnisse - Accuracy

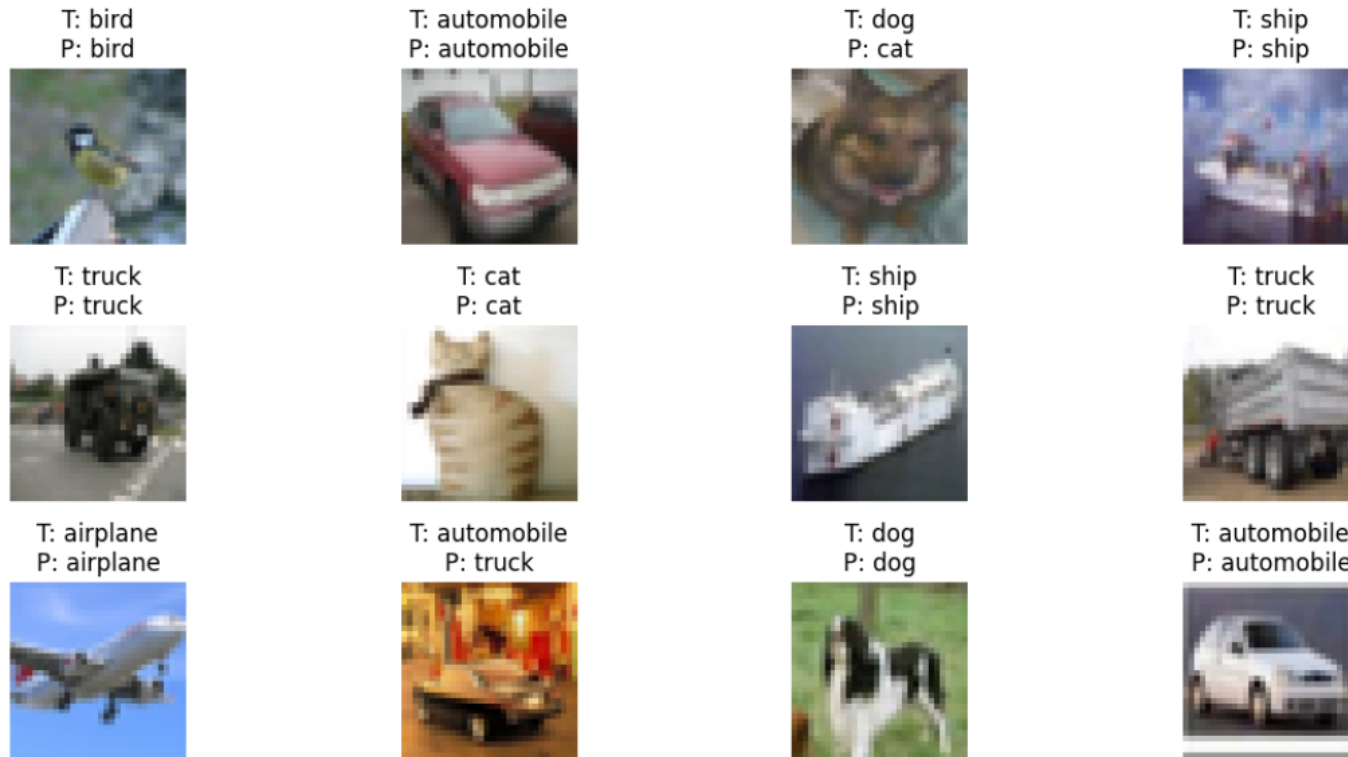
```
plt.figure(figsize=(8,5))
plt.plot(hist["accuracy"], marker="o", label="Train Accuracy")
plt.plot(hist["val_accuracy"], marker="o", label="Val Accuracy")
plt.title("Accuracy über Epochen")
plt.xlabel("Epoche")
plt.ylabel("Accuracy")
plt.legend()
plt.tight_layout()
plt.show()
```

- Plotten der Accuracy Werte ebenfalls über 10 Epochen
- Die Trainings- und Validierungswerte liegen nah beieinander, was auf kein starkes Overfitting hindeutet.



Ergebnisse - Beispielvorhersagen

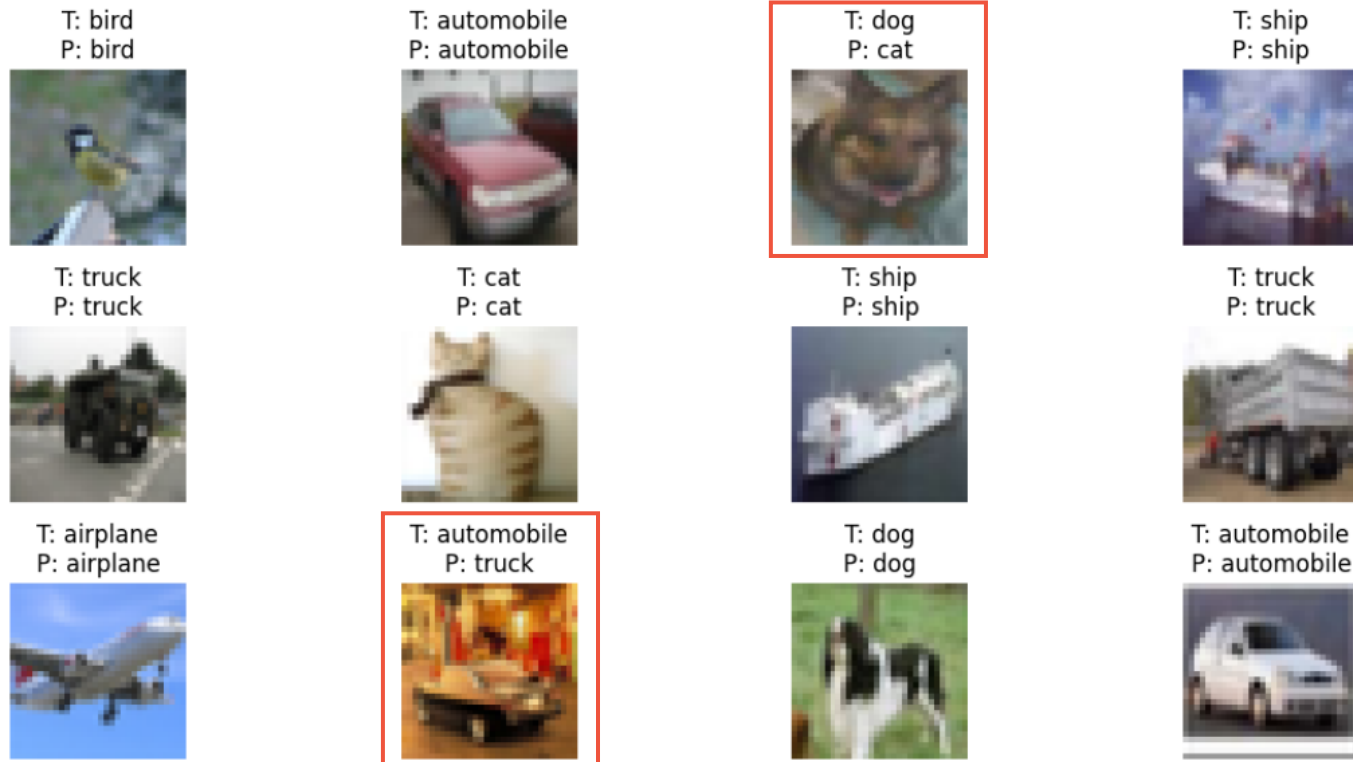
Beispielvorhersagen (Test)



- T = Wahr (engl. *True*)
 - P = Vorhersage (engl. *Predict*)
- Abbildung zeigt, dass das Modell bereits viele Klassen korrekt erkennt, insbesondere bei klaren Bildstrukturen.
- Aber: Fehlklassifikationen treten vor allem bei visuell ähnlichen Klassen auf.

Ergebnisse - Fehlklassifikationen

Beispielvorhersagen (Test)



Ursachen für Fehlklassifikationen?

- geringe Bildauflösung (32x32 Pixel)
- überlappende visuelle Merkmale
→ Stützung durch Konfusionsmatrix

Beispiele

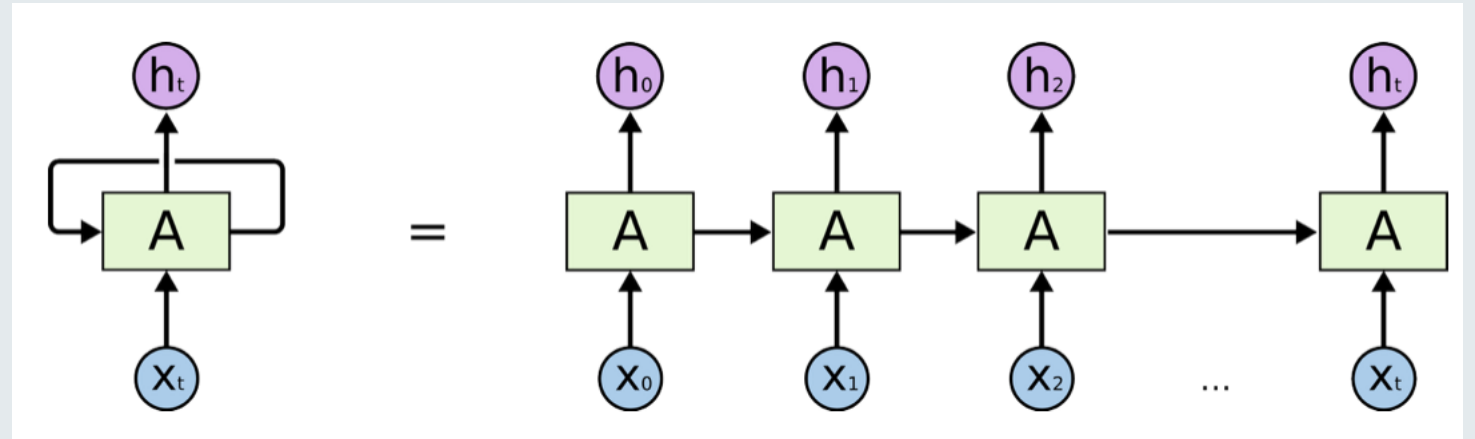
- cat vs. dog
- truck vs. automobile

Zeitreihenanalyse mit RNN

Insbesondere mit einem Long Short-Term Memory Network

Was ist ein RNN?

- Für sequenzielle Daten gemacht, bei denen die Reihenfolge entscheidend ist
- RNN hat ein Gedächtnis, Informationen werden von einem Schritt zum nächsten gegeben
- Problem: Vanishing Gradient



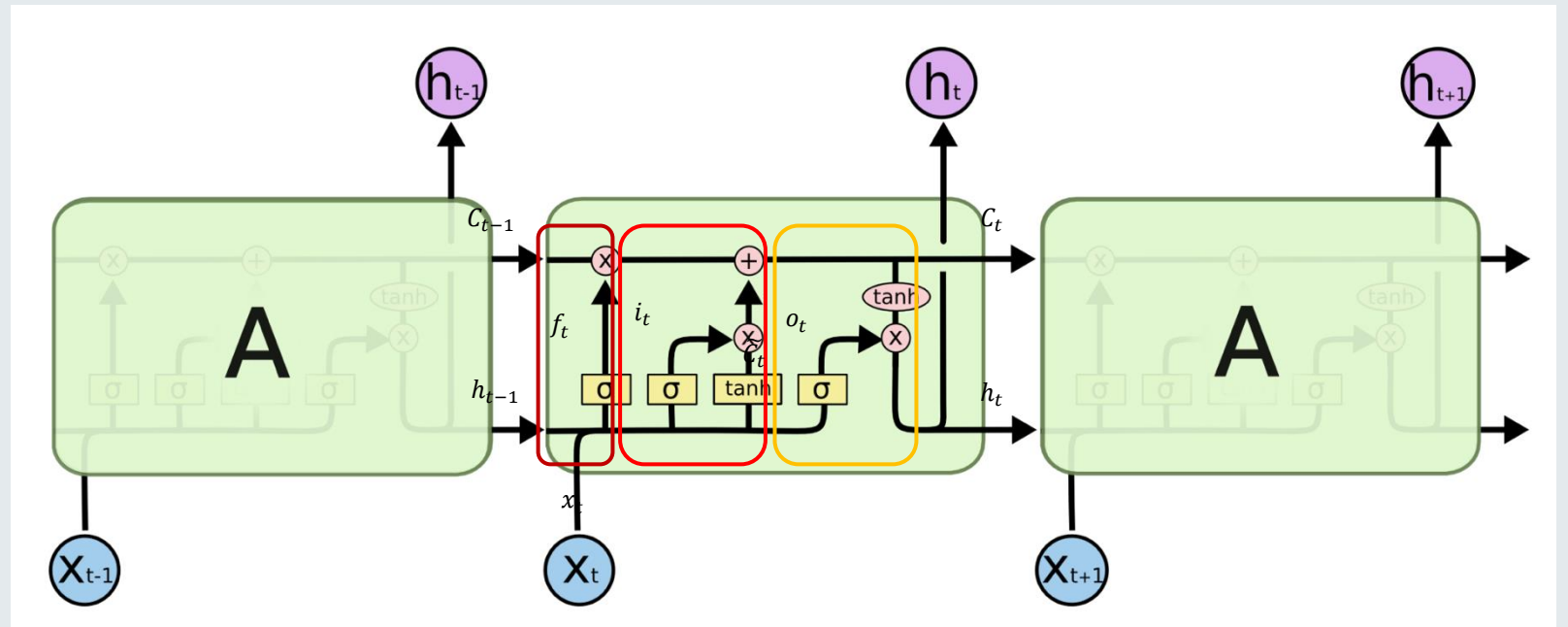
→ Die Lösung ist das LSTM

Was ist ein LSTM?

Forget Gate $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$

Input Gate $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$
 $\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$
 $C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$

Output Gate $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$
 $h_t = o_t * \tanh(C_t)$



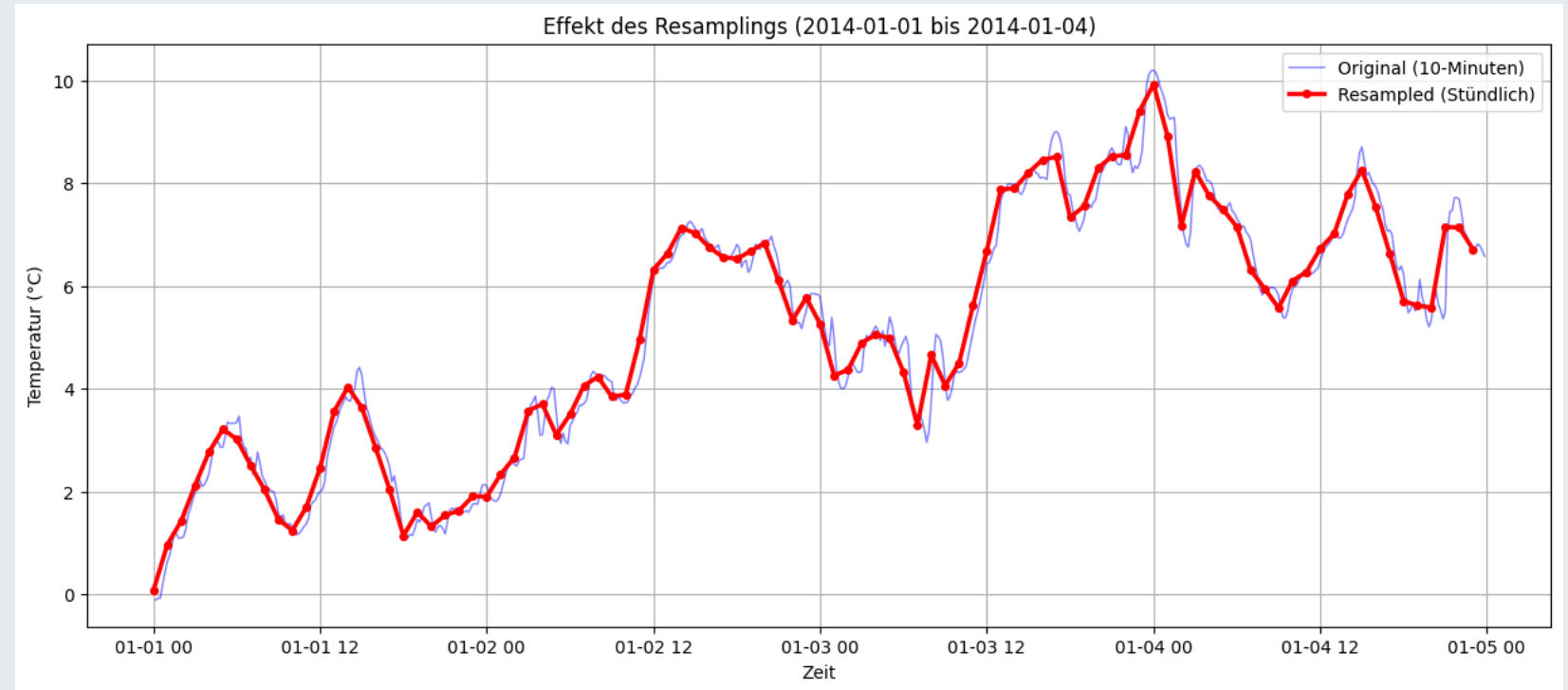
Laden des Jena Climate Datensatzes

- Über 150.000 Datenzeilen
- Messungen in einem 10 Minuten Intervall
- 21 Features

```
1 import os
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 ordnerName = 'Datensätze'
6 dateiNamen = [
7     'mpi_roof_2014a.csv',
8     'mpi_roof_2014b.csv',
9     'mpi_roof_2015a.csv',
10    'mpi_roof_2015b.csv',
11    'mpi_roof_2016a.csv',
12    'mpi_roof_2016b.csv'
13 ]
14
15 datensatzArray = []
16
17 for datensatzName in dateiNamen:
18     pfadName = os.path.join(ordnerName, datensatzName)
19     print(f"Lade Datei: {pfadName}")
20     datensatzArray.append(pd.read_csv(pfadName, encoding='ISO-8859-1'))
21
22 datensatz = pd.concat(datensatzArray, ignore_index = True)
23 print("Datensätze erfolgreich zusammengefügt")
24 display(datensatz)
```

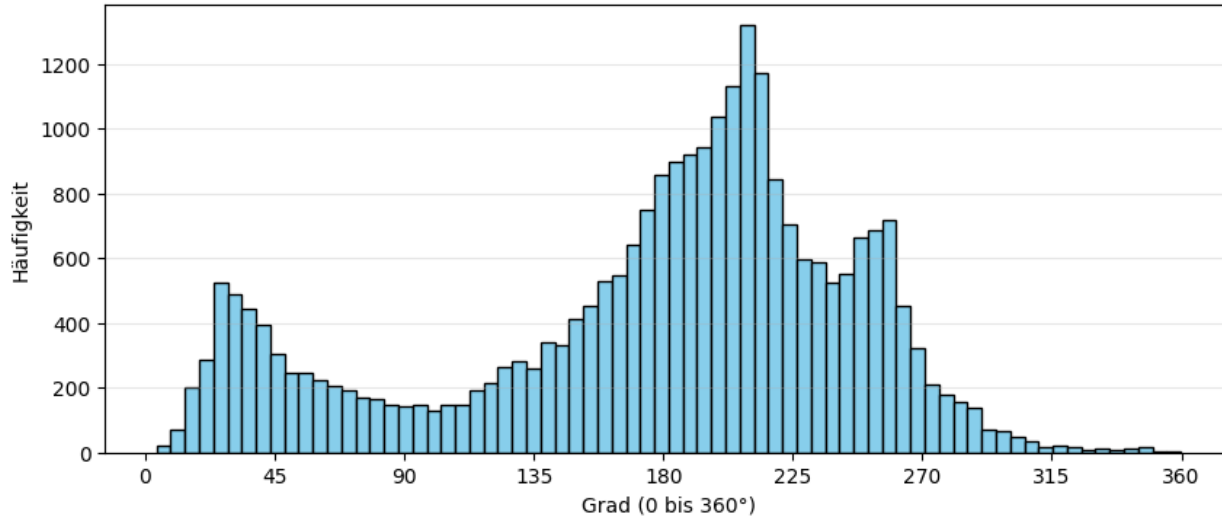
Resampling

- **Ausgangslage:** Daten liegen im 10-Minuten-Takt vor
- **Verarbeitung:** Downsampling auf stündliche Mittelwerte
- **Vorteil:** Reduktion der Datenmenge und Glättung von Messfehlern



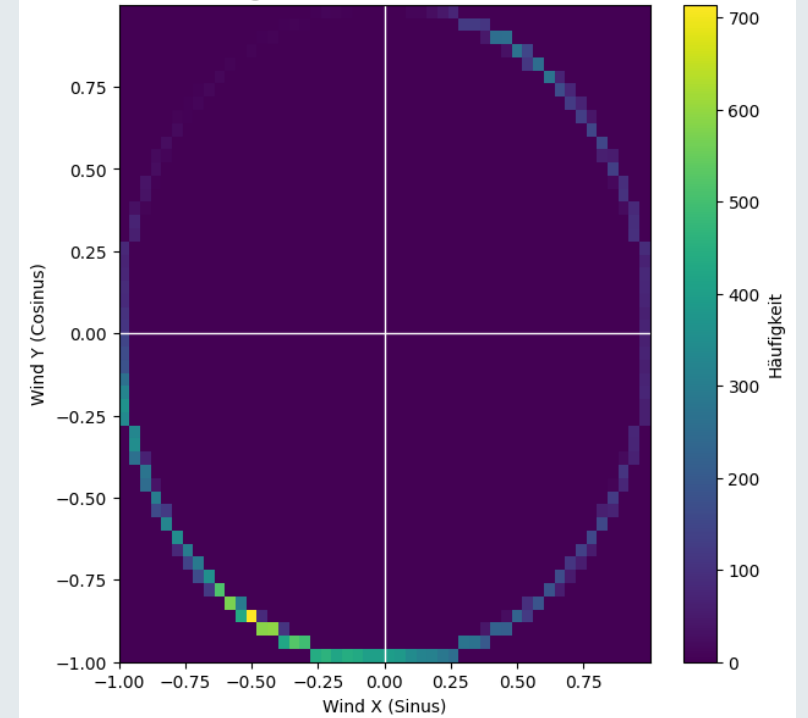
Transformation der Wind Daten

Windrichtung in Grad (Vor der Transformation)

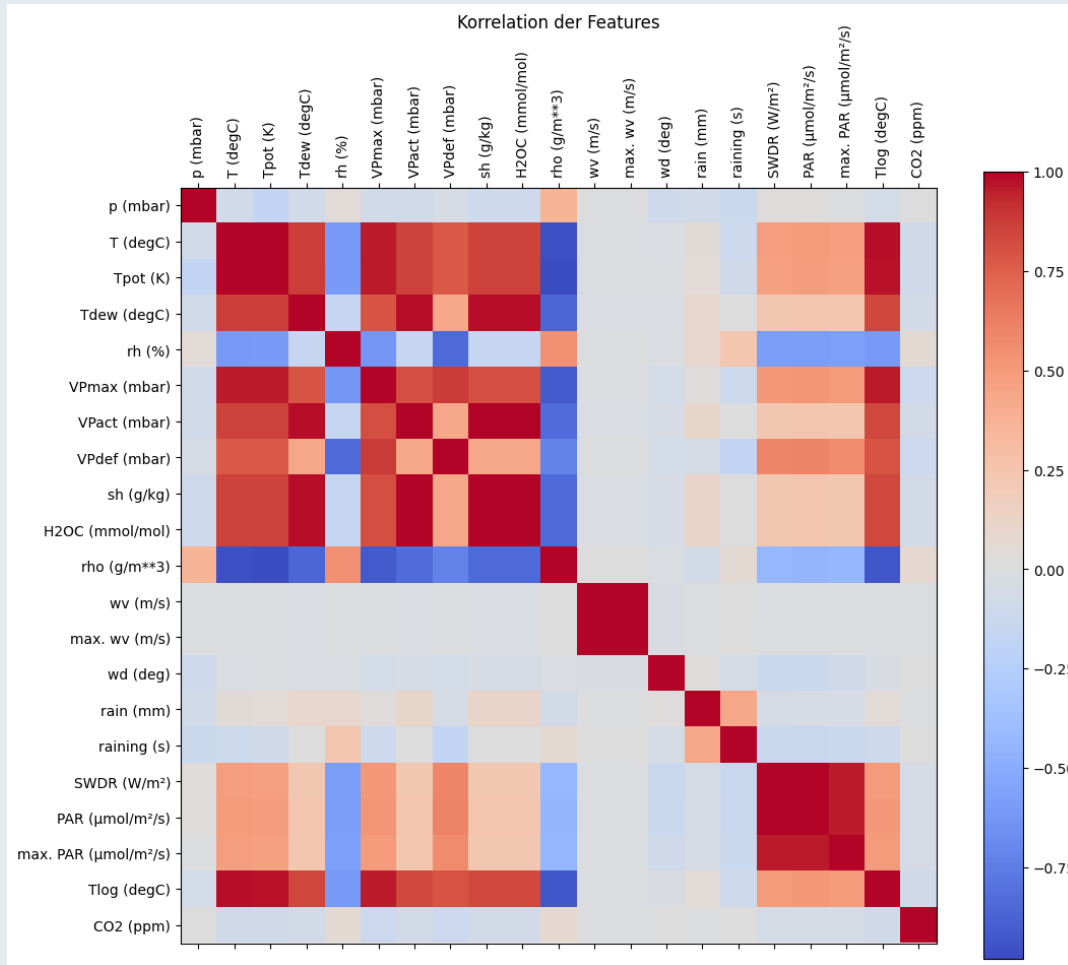


- **Problem:** 0° und 360° sind identisch, aber numerisch weit entfernt
- **Lösung:** Umwandlung der Windrichtung in Vektoren (W_x & W_y) mittels Sinus und Cosinus.
- **Ergebnis:** Eine stetige, zyklische Funktion ohne harte Brüche, die das Modell leicht lernen kann.

Windrichtung als Vektor (Nach der Transformation)



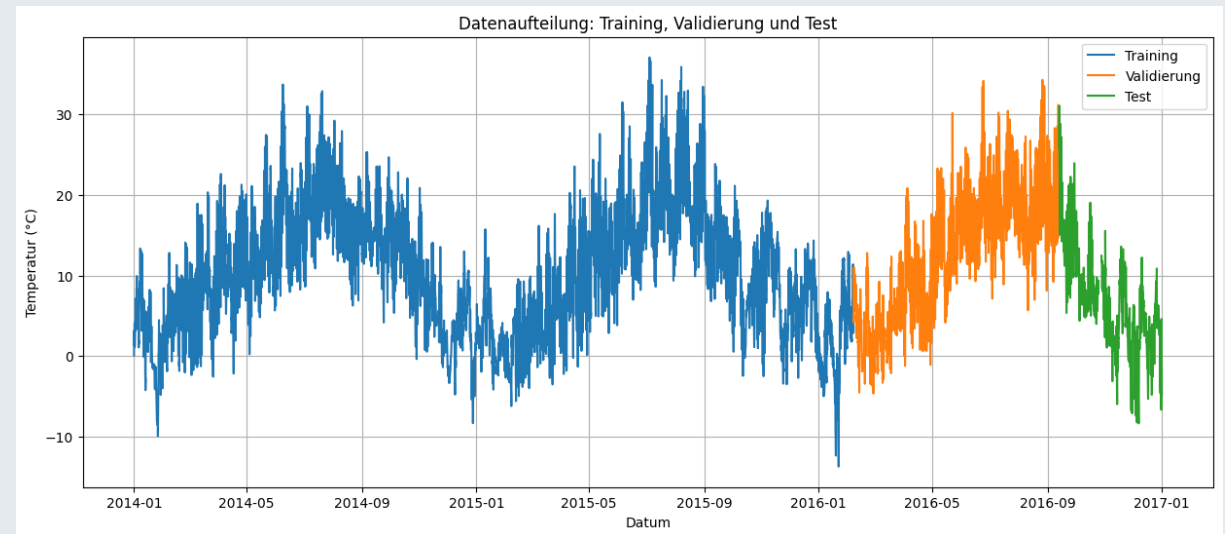
Entfernung redundanter Features



- **Analyse:** Identifizierung redundanter (doppelter) Informationen durch Korrelationsanalyse
- **Erkenntnis (Beispiel):** Temperatur (T) und Potentielle Temperatur (Tpot) korrelieren zu fast 100%
- **Maßnahme:** Löschen von Tpot, rho, H2OC etc., um das Modell effizienter zu machen und "Ballast" abzuwerfen

Datenaufbereitung

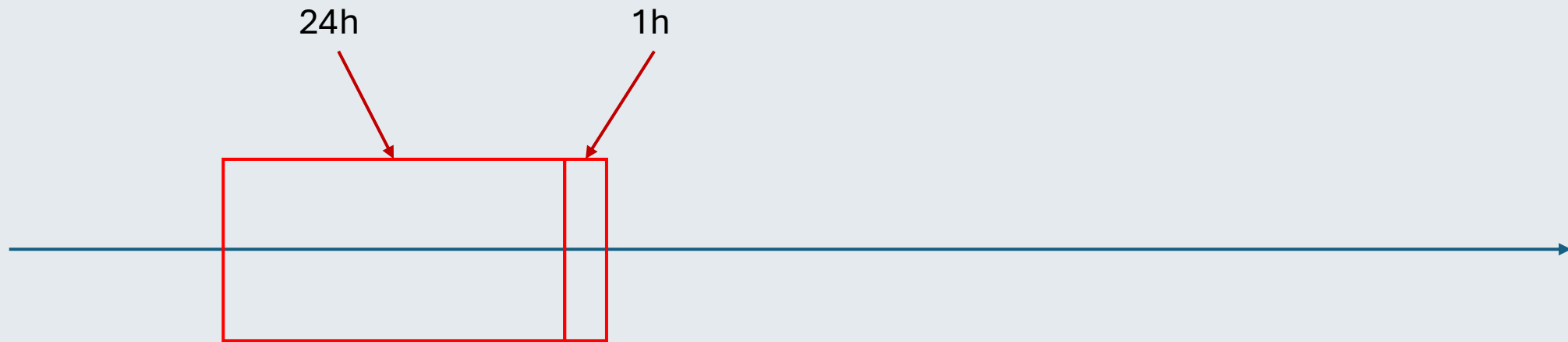
- **Normalisierung:** Skalierung aller Werte (Standardabweichung), damit das LSTM stabil lernt.
- **Split:** Aufteilung in Training (70%), Validierung (20%) und Test (10%).



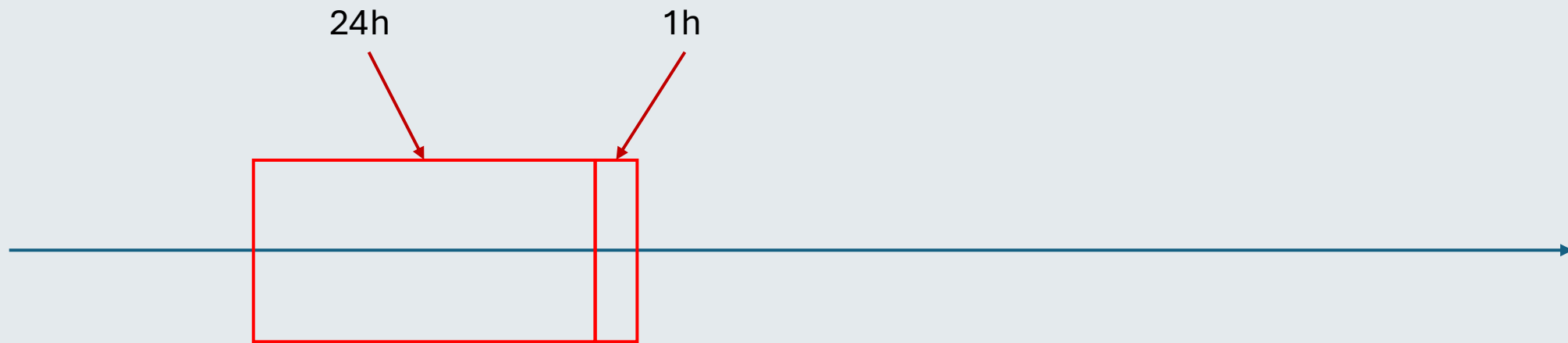
Das „Sliding Window“ Verfahren

- **Input:** Das Modell sieht immer exakt die letzten 24 Stunden
- **Target:** Das Ziel ist der Wert der nächsten Stunde (Forecast Horizon = 1)
- **Batching:** Das Fenster gleitet Schritt für Schritt über den gesamten Datensatz

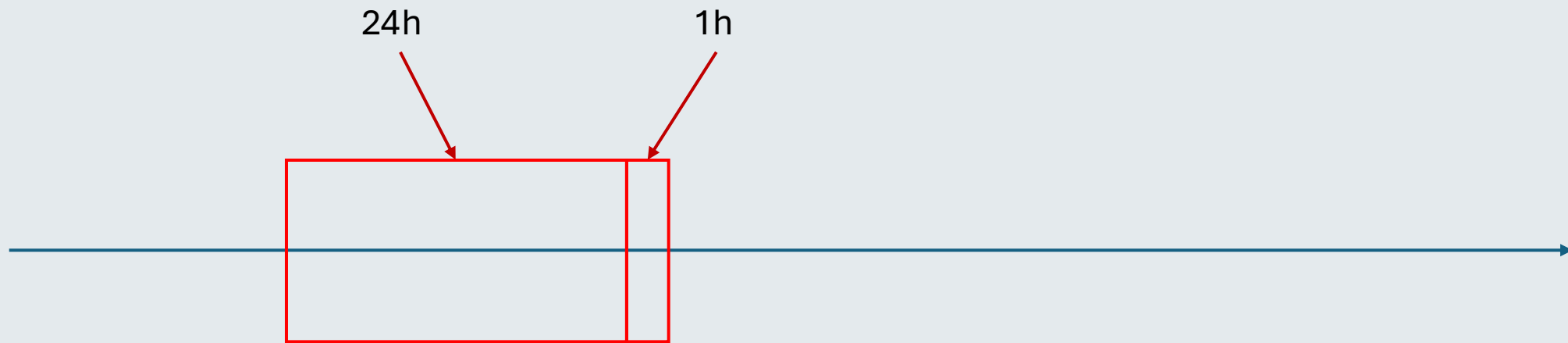
Das „Sliding Window“ Verfahren



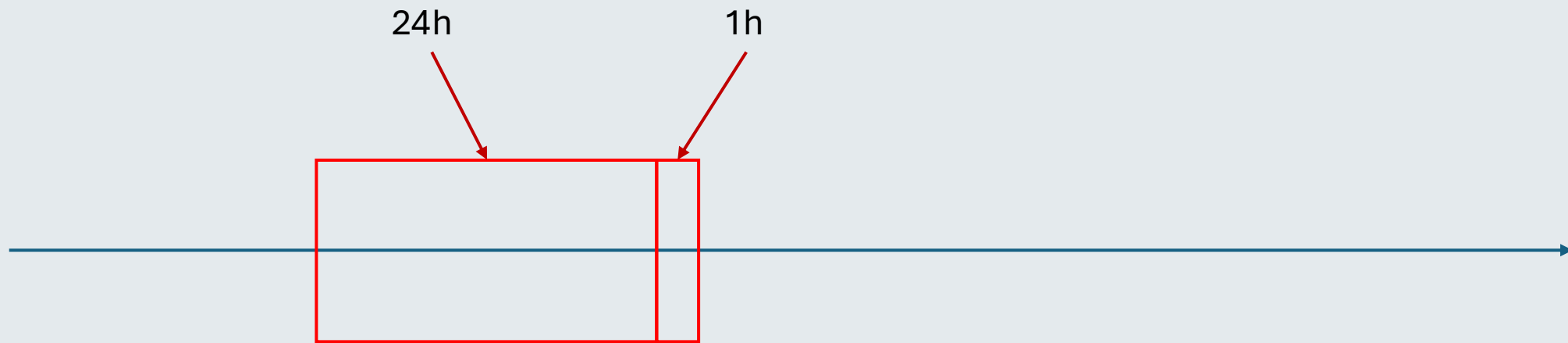
Das „Sliding Window“ Verfahren



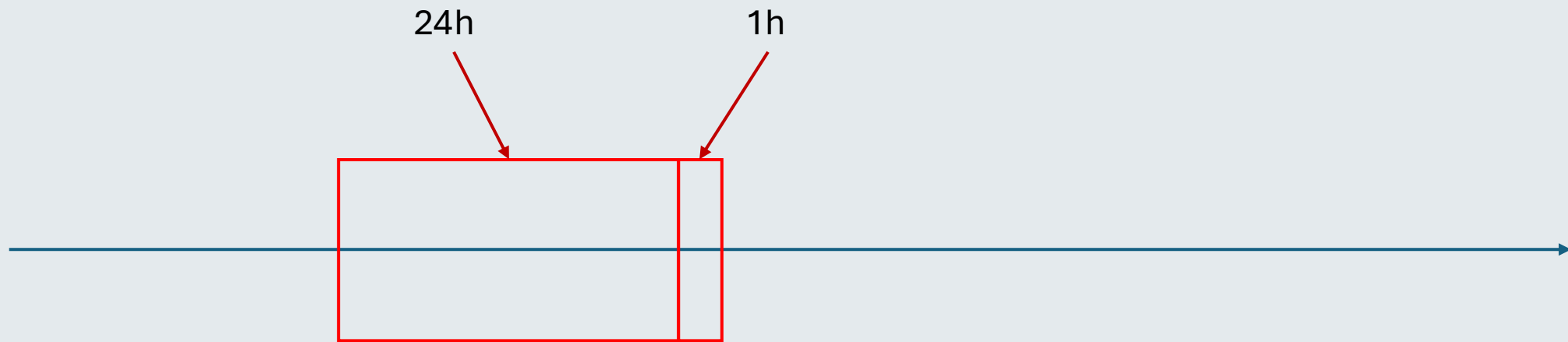
Das „Sliding Window“ Verfahren



Das „Sliding Window“ Verfahren



Das „Sliding Window“ Verfahren



Die Modell-Architektur

- **LSTM Layer (32 Units):** Erfasst zeitliche Abhängigkeiten und speichert "Zustände" über 24 Stunden
- **Dropout (20%):** Zufälliges Abschalten von Neuronen im Training zur Vermeidung von Auswendiglernen (Overfitting)
- **Dense Layer:** Führt die 32 Merkmale zusammen zu einem einzigen Ausgabewert (Temperatur)

```
1 # Schicht 1
2 modell.add(layers.LSTM(units = 32, input_shape = (FENSTER_GROESSE, anzahlSpalten)))
3
4 # Schicht 2
5 modell.add(layers.Dropout(0.2))
6
7 # Schicht 3
8 modell.add(layers.Dense(units = 1))
9
```

Training

- Nutzung des Adam-Optimizer
- Loss: MAE
- Genauigkeit: Der durchschnittliche Fehler liegt bei 0.34 °C

```
1  modell.compile(  
2      loss = 'mae',  
3      optimizer = 'adam',  
4      metrics = ['mae'],  
5  )  
6  
7  modell.summary()  
8  
9  print("Training starten ...")  
10  
11 verlauf = modell.fit(  
12     trainingsDatensatzFenster,  
13     epochs = 10,  
14     validation_data = validierungsDatensatzFenster,  
15 )
```


Hyperparameter Tuning

gewählte Hyperparameter:

```
# Basismodell  
  
layer.LSTM(units=32)  
FENSTER_GROESSE = 24  
layers.Dropout(0.2)
```

Ergebnis des Basismodells:

Test MAE → 0,04557 ~ 4,6%

- `layer.LSTM(...)`
 - verarbeitet zeitliche Sequenzen und lernt Abhängigkeiten zwischen aufeinanderfolgenden Zeitschritten.
- `FENSTER_GROESSE`
 - wie viele vergangene Zeitschritte stehen dem Modell gleichzeitig als Eingabe zur Verfügung.
- `layers.Dropout(...)`
 - deaktiviert während des Trainings zufällig einen Anteil der Neuronen, zur Vermeidung von Overfitting.

Hyperparameter Tuning – Variante 1

```
# Variante 1  
  
layer.LSTM(units=64) # von 32 auf 64 erhöht  
FENSTER_GROESSE = 24  
layers.Dropout(0.2)
```

- Erhöhung der Units in layer.LSTM(...) von 32 auf 64.
- Der Rest bleibt unverändert.

Was ist das Ergebnis?

Ergebnisse des Basismodells

Test MAE → 0,04557 ~ 4,6%

Ergebnisse der 1. Variante

Test MAE → 0,04206 ~ 4,2% | $\searrow$ 0,351%

Hyperparameter Tuning – Variante 2

```
# Variante 2

layer.LSTM(units=32)
FENSTER_GROESSE = 48 # von 24 auf 48 gesteigert
layers.Dropout(0.2)
```

- Steigerung der FENSTER_GROESSE von 24 auf 48.
- Der Rest bleibt unverändert.

Was ist das Ergebnis?

Ergebnisse des Basismodells

Test MAE → 0,04557 ~ 4,6%

Ergebnisse der 2. Variante

Test MAE → 0,04994 ~ 5% | ↗ 0,437%

Hyperparameter Tuning – Variante 3

```
# Variante 3  
  
layer.LSTM(units=32)  
FENSTER_GROESSE = 24  
layers.Dropout(0.4) # Anpassung von 0.2 auf 0.4
```

- Anpassung der layers.Dropout(...) von (0.2) auf (0.4).
- Der Rest bleibt unverändert.

Was ist das Ergebnis?

Ergebnisse des Basismodells

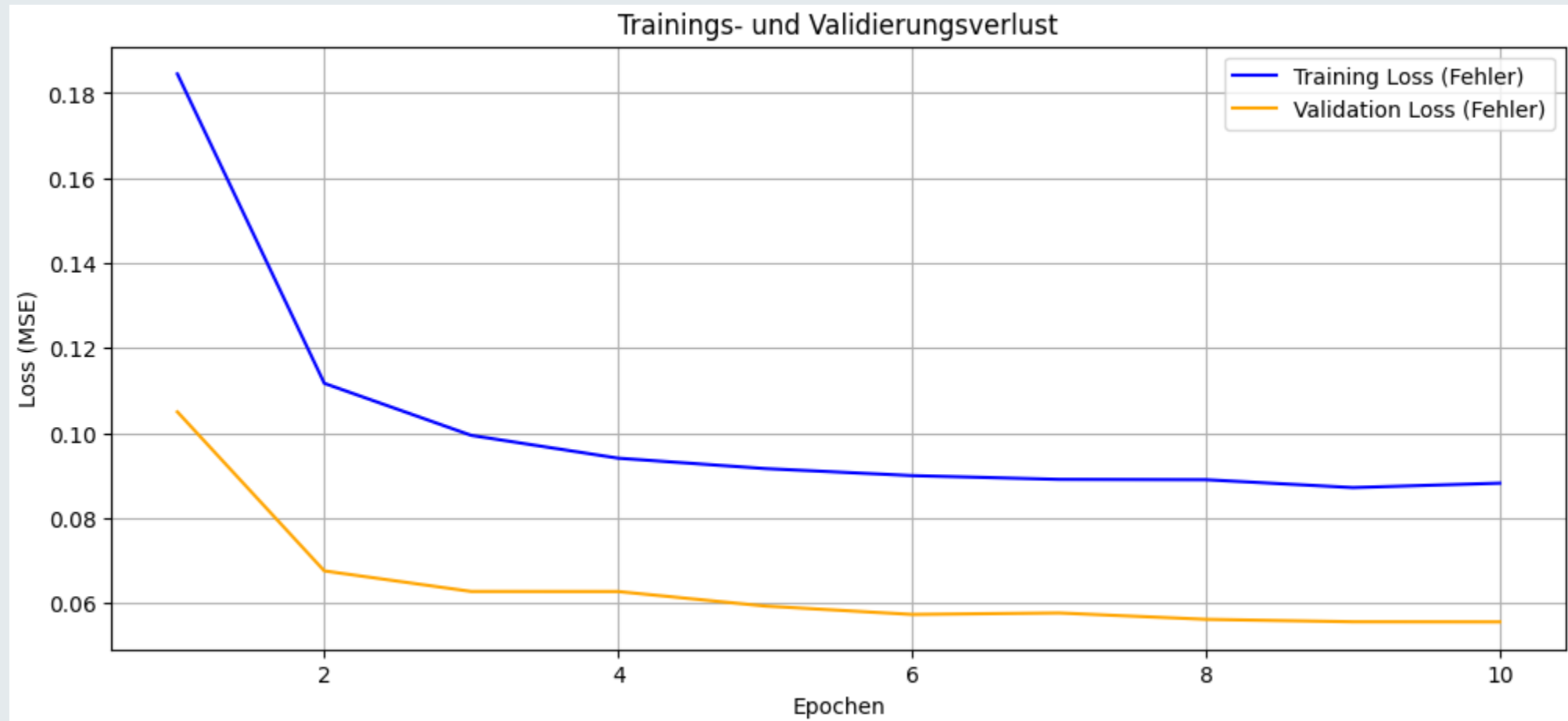
Test MAE → 0,04557 ~ 4,6%

Ergebnisse der 3. Variante

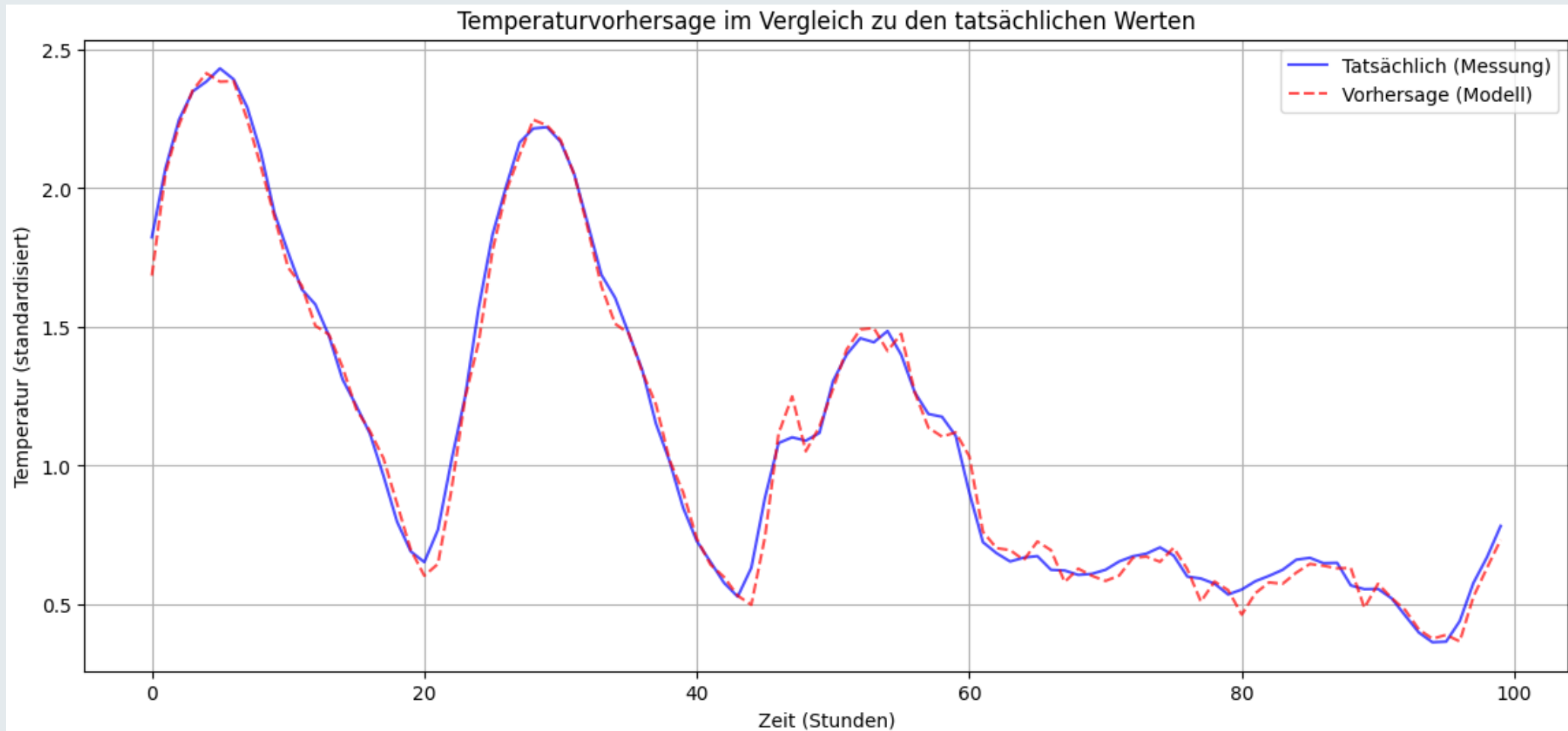
Test MAE → 0,04597 ~ 4,6% | ↗ 0,04%

Visualisierung der Ergebnisse

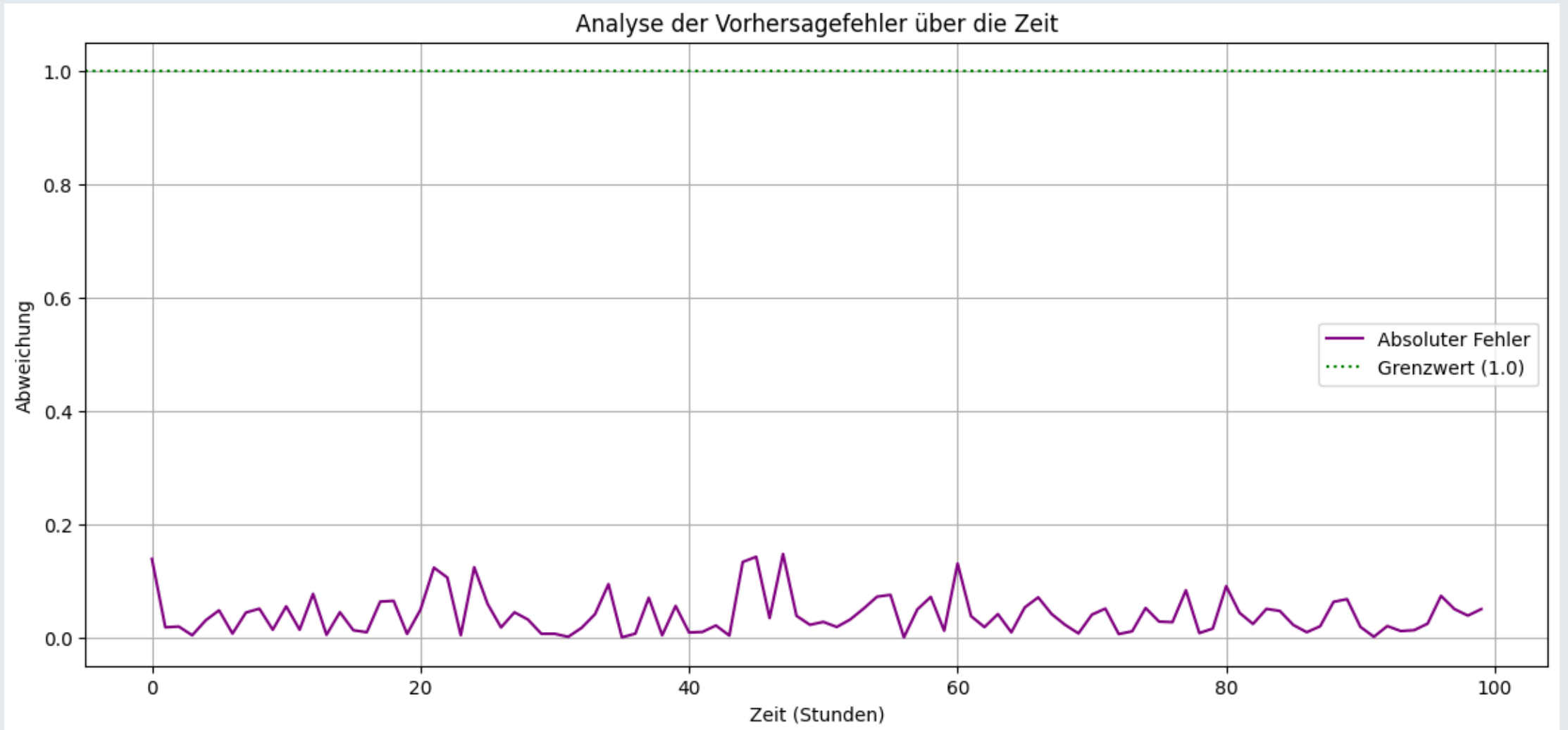
Visualisierung der Ergebnisse



Visualisierung der Ergebnisse



Visualisierung der Ergebnisse



Pipeline

Pipeline

- Konzeption einer ML-Pipeline
- Technologie-Stack und Tools
- Automatisierung der Pipeline
- Evaluierung und Monitoring
- Visualisierung der Ergebnisse

Konzeption einer ML-Pipeline

- Ein vollautomatisierter Workflow, der Rohdaten in ein nutzbares Vorhersagemodell verwandelt.
- Zentrale Ziele:
 - Reproduzierbarkeit: Gleiche Daten führen immer zum exakt gleichen Modell.
 - Skalierbarkeit: Prozess funktioniert stabil, egal ob 1.000 oder 1 Mio. Datensätze.
 - Qualitätssicherung: Vermeidung des "Garbage In, Garbage Out"-Prinzips durch Standardisierung.

Konzeption einer ML-Pipeline

- Schritte einer Pipeline

- Datenaufbereitung
- Modelltraining
- Automatisierung
- Monitoring & Evaluierung der Pipeline
- Visualisierung der Ergebnisse

- Zusammenhang

- Ohne Datenaufbereitung → kein sinnvolles Training
- Ohne Training → keine Evaluierung
- Ohne Evaluierung → keine Aussage über Modellqualität
- Ohne Automatisierung → keine Reproduzierbarkeit
- Ohne Monitoring → keine Stabilität
- Ohne Visualisierung → kein nachvollziehbares Ergebnis

Konzeption einer ML-Pipeline



Konzeption einer ML-Pipeline



```
1 import pandas as pd
2 import numpy as np
3 import joblib
4
5 from sklearn.model_selection import train_test_split
6 from sklearn.pipeline import Pipeline
7 from sklearn.compose import ColumnTransformer
8 from sklearn.preprocessing import StandardScaler, OneHotEncoder
9 from sklearn.ensemble import RandomForestRegressor
10 from sklearn.impute import SimpleImputer
11 from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
12 from sklearn.model_selection import cross_val_score
13
14
15
16
17 data = pd.read_csv('vehicle_emissions.csv')
18
19 X = data.drop(["CO2_Emissions"], axis=1)
20 y = data["CO2_Emissions"]
```

Konzeption einer ML-Pipeline

```
1 numerical_cols = ["Model_Year", "Engine_Size", "Cylinders", "Fuel_Consumption_in_City(L/100 km)", "Fuel_Consumption_in_City_Hwy(L/100 km)", "Fuel_Consumption_comb(L/100km)", "Smog_Level"]
2 categorical_cols = ["Make", "Model", "Transmission", "Vehicle_Class"]
3
4 numerical_pipeline = Pipeline([
5     ('imputer', SimpleImputer(strategy="mean")),
6     ('scaler', StandardScaler())
7 ])
8
9 categorical_pipeline = Pipeline([
10    ('imputer', SimpleImputer(strategy="most_frequent")),
11    ('encoder', OneHotEncoder(handle_unknown='ignore'))
12 ])
13
14 preprocessor = ColumnTransformer([
15     ('num', numerical_pipeline, numerical_cols),
16     ('cat', categorical_pipeline, categorical_cols)
17 ])
18
19
20 pipeline = Pipeline([
21     ('preprocessor', preprocessor),
22     ('regressor', RandomForestRegressor())
23 ])
```

Konzeption einer ML-Pipeline



```
1  scores = cross_val_score(pipeline, X, y, cv=5)
2
3  print(scores)
4
5  X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
6
7  pipeline.fit(X_train, y_train)
8
9  prediction = pipeline.predict(X_test)
10
11 mse = mean_squared_error(y_test, prediction)
12 mae = mean_absolute_error(y_test, prediction)
13 r2 = r2_score(y_test, prediction)
14 rsme = np.sqrt(mse)
15
```


Konzeption einer ML-Pipeline

```
1 train_prediction = pipeline.predict(X_train)
2
3
4 mse_train = mean_squared_error(y_train, train_prediction)
5 mae_train = mean_absolute_error(y_train, train_prediction)
6 r2_train = r2_score(y_train, train_prediction)
7 rmse_train = np.sqrt(mse_train)
8
9 print(f"Training R-squared: {r2_train}")
10 print(f"Test R-squared: {r2}")
11
12 print(f"Mean Squared Error: {rmse}")
13 print(f"Mean Absolute Error: {mae}")
14 print(f"R-squared: {r2}")
15
16
17
18
19 joblib.dump(pipeline, 'co2_model.joblib')
```

Technologie-Stack und Tools

- Sklearn
- Panda
- Numpy
- Joblib
- GitHub Actions
 1. Nahtlose Integration (Single Source of Truth)
 2. Kein Wartungsaufwand (Zero Maintenance Overhead)
 3. Infrastruktur als Code
- Andere Optionen
 1. Jenkins
 2. Apache Airflow
 3. Etc.

Automatisierung der Pipeline

Workflow file for this run

[.github/workflows/pipeline.yml](#) at 61f683e

```
1  name: ML Pipeline
2
3  on:
4    push:
5      branches: [ "main" ]
6    workflow_dispatch:
7
8  jobs:
9    build-and-run:
10     runs-on: ubuntu-latest
11
12     steps:
13       - name: Checkout Code
14         uses: actions/checkout@v4
15
16       - name: Set up Python
17         uses: actions/setup-python@v5
18         with:
19           python-version: '3.9'
20
21       - name: Install Libraries
22         working-directory: "Projektarbeit/3_PIPELINE"
23         run: |
24           pip install -r requirements.txt
25
26       - name: Run Script
27         working-directory: "Projektarbeit/3_PIPELINE"
28         run: python Pipeline.py
```

Automatisierung der Pipeline

- Pipeline wird Automatisch bei jedem Push ausgeführt
 - Ermöglicht Continues Integration (CI)
- Nutzt immer einen Frischen Server
 - Verhindert „Works on my machine“-Fehler durch neutrale Cloud-Umgebung
- Automatisches Klonen des Repository
- Bereitstellung einer sauberen Python Laufzeitumgebung
- Installation aller ML-Bibliotheken
- Start des Skripts

Vorteile der Pipeline

- Reproduzierbarkeit
 - Deterministische Umgebungen
 - Versionierung von Code und Daten
- Modularität
 - Numerische und kategoriale Daten werden getrennt behandelt
- All-in-One Modell
 - Mögliches gespeichertes Modell enthält nicht nur Ergebnis sondern auch den gesamten Vorbereitungsweg
 - Daten müssen später nicht nochmal durchgearbeitet werden
- Automatisierung
 - Continuous Integration
 - Standardisierte Umgebung (Ubuntu-Container)

Herausforderungen

- Kategorische Hürden
 - Wenn im Testset eine Automarke vorkommt, die im Training nicht vorkam, könnte ein Fehler geworfen werden
- Komplexität der .yaml Datei
 - Syntax Sensitivität
- Pfad-Problematik
 - Ordnerstruktur muss eingehalten werden
- Lange Iterationszyklen

Monitoring

- Ergebnisse vom Data Drift Monitoring
 - Kolmogorov-Smirnov-Test (K-S-Test)
 - Geprüft wird kritische Veränderung von Features

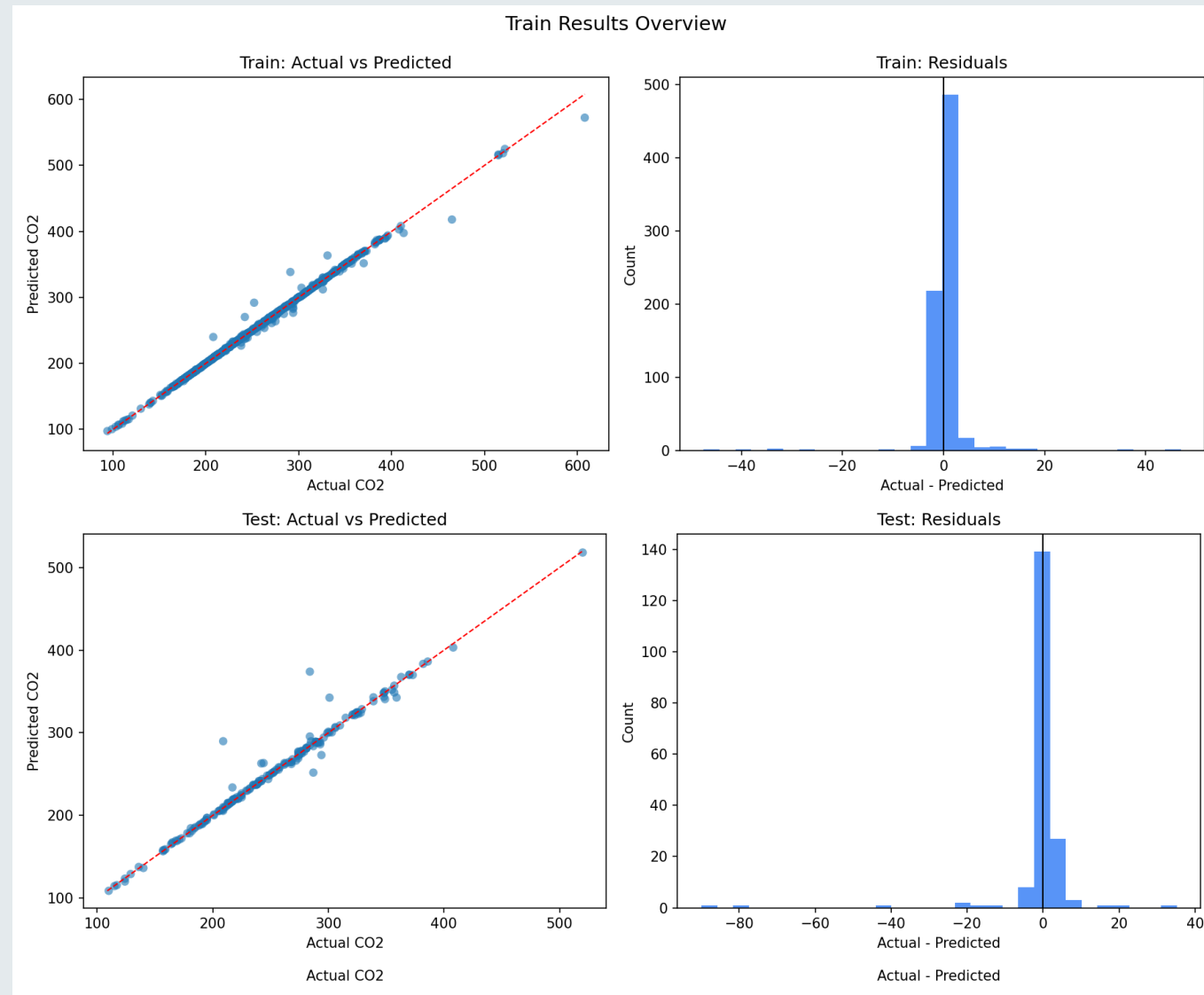
Weitere Metriken zum Monitoring:

- Data Quality
- Concept Drift

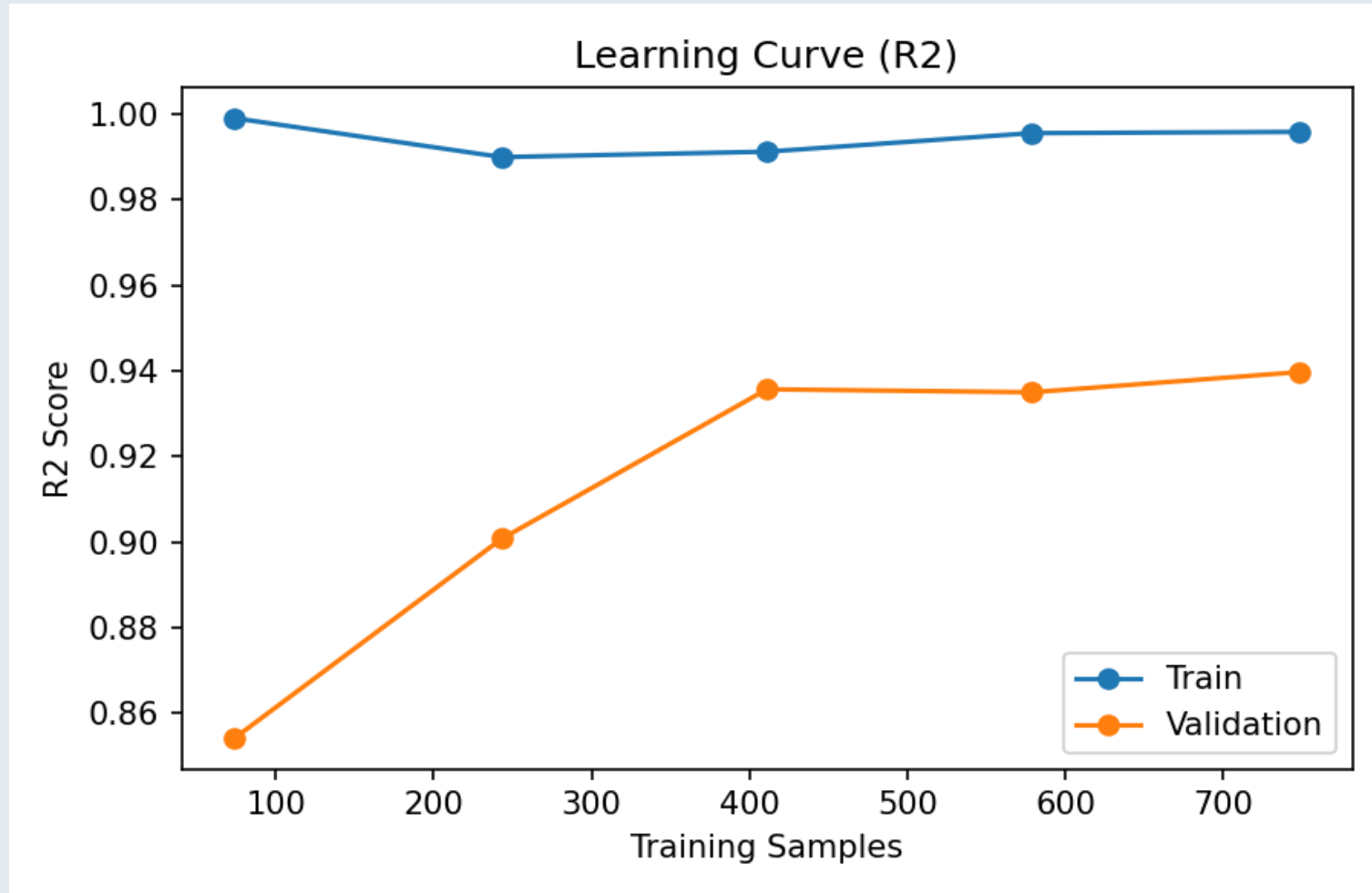
```
--- Starte Daten-Monitoring ---  
Prüfe Merkmal: Engine_Size  
  Abstandswert: 0.0428, Wahrscheinlichkeit (P-Wert): 0.9304  
  [OK] Keine nennenswerte Änderung.  
Prüfe Merkmal: Fuel_Consumption_comb(L/100km)  
  Abstandswert: 0.3818, Wahrscheinlichkeit (P-Wert): 0.0000  
WARNUNG: Signifikanter Data Drift bei Fuel_Consumption_comb(L/100km) erkannt
```

Visualisierung der Ergebnisse

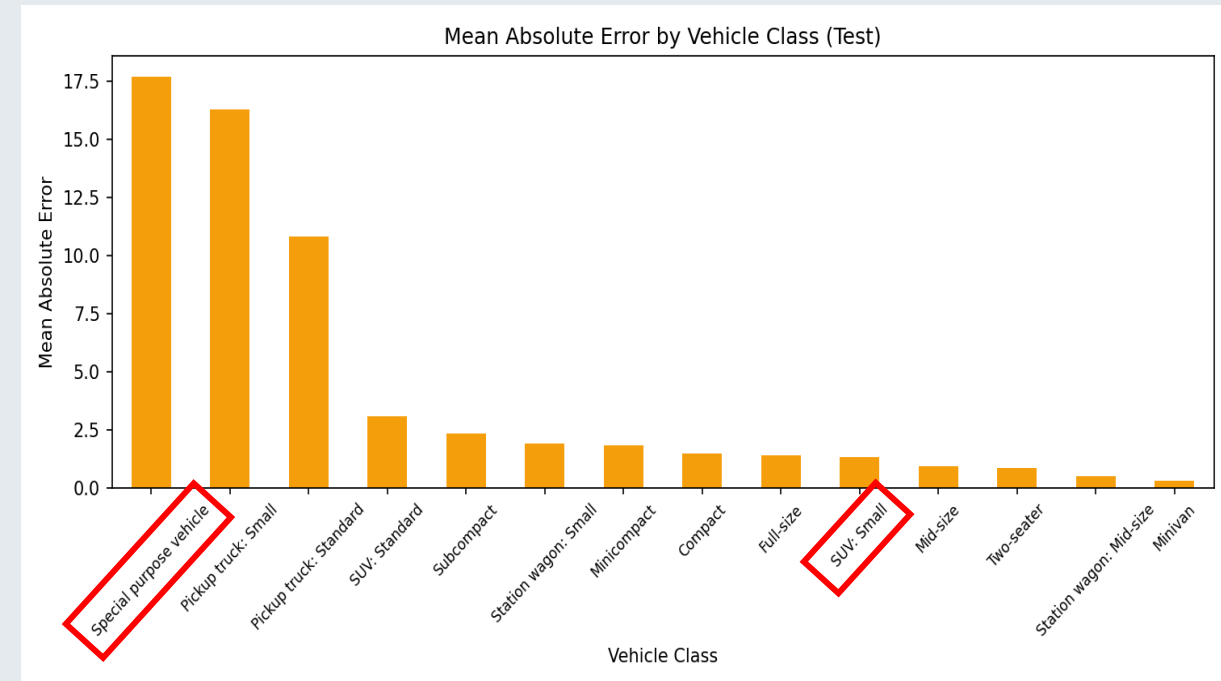
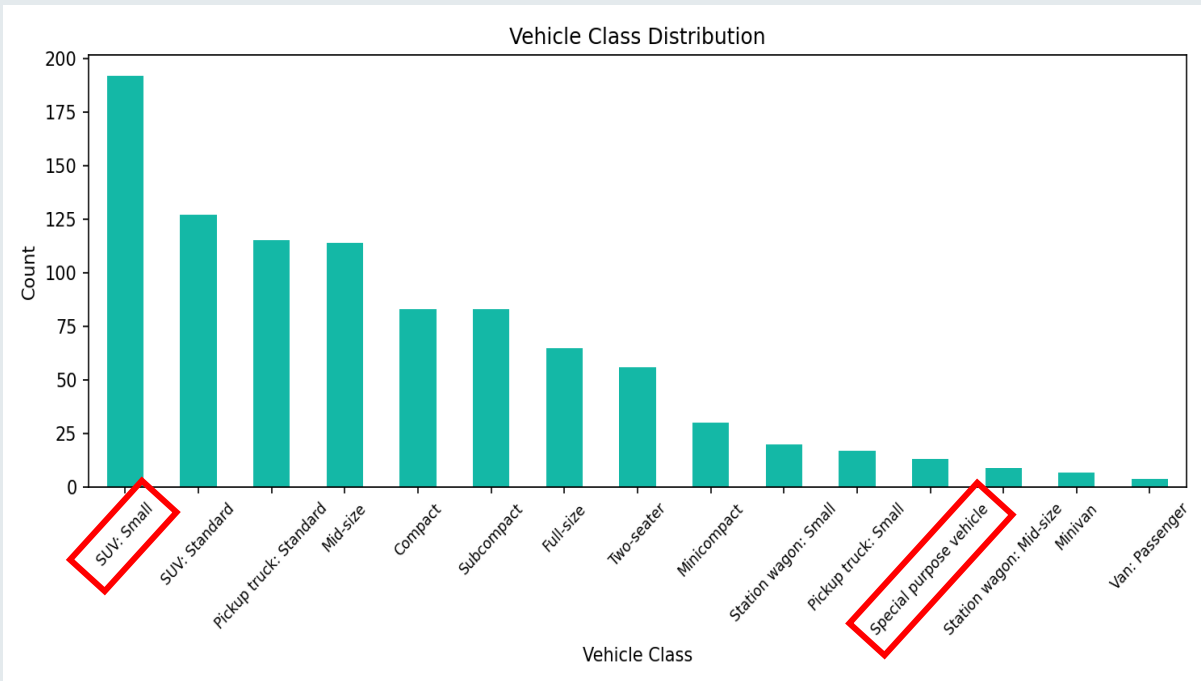
Visualisierung



Visualisierung



Visualisierung



Quellenverzeichnis

- CNN Aktivierungsfunktionen
- CNN 1
- CNN 2
- What is a Machine Learning Pipeline? – GeeksforGeeks
- ML-Pipelines | Machine Learning | Google for Developers
- Konfusionsmatrix
- LSTM
- Backpropagation
- Tensorflow LSTM